Aran: The Full Project Story, From Zero to Now

19 October 2026 · For Inbathamizhan S (lead) and the Aran team: Aasif Ahmed A, SN Arun Vijay, Bharadhwaj M · KARE

> **What this is.** One document that explains the whole Aran project from the very first message to today: what the problem is, every technical word you need, what research was done and how, which papers were used and what each one gave us, how the solution was derived step by step, what was built, how it was built, what every experiment measured, what is still missing, and what comes next. It was written after an audit of the whole repository (`main` at commit `1554442`), every research report, the 20 learning docs, the Obsidian notes, and every project thread from 16 to 17 October (IST).
>
> **Who should read it.** Everyone on the team, before presenting or answering questions about Aran. You do not need to have read anything else first.

How to read this document

The document has eight parts. Read them in order the first time.

Part	What it gives you	Time
Part A. Technical foundation	Every technical word used in the project, explained in plain language with an Aran example. Read this first, then come back to it whenever a word is unclear.	60 to 90 min
Part B. The story, from zero	What happened, in order: the first message, the research, each build, each decision, up to today.	45 min
Part C. The research: papers, gaps and how the solution was derived	Exactly what research was done, which papers and rules were used, how each one was used, and the chain of reasoning that led to Aran's design.	45 min
Part D. How Aran works, end to end	The architecture: one real request followed through every layer, then each part in depth.	60 min
Part E. Every experiment and what it proved	E1 to E19: what was measured, on what data, the result, and what it means. Also every honest weakness.	30 min
Part F. The live console and the demo	The website that runs Aran's real code: every page, the guided tour, the stand-out demo features, how to run it.	20 min
Part G. What is not done, and the future	Open items, the roadmap, the product vision and a plan per pair.	20 min

Part	What it gives you	Time
Part H. Explaining Aran to others	A 30-second pitch, a 3-minute version, and answers to the questions judges will ask.	15 min

Appendices at the end list every file in the repository, the learning-doc reading order, how the team worked, and exactly what is real versus generated or simulated.

One honest note before starting. This document separates three kinds of statement. *Measured* means a script ran and the number is in a results file in `research/`. *Designed* means it is in the research document with reasoning or a proof, but not yet built. *Assumed* means a number we chose (for example how many years an Aadhaar number stays sensitive). When a sentence does not say which, it is measured.

Part A. Technical foundation

This part explains every technical word that appears in Aran. Each entry has a plain meaning and, where it helps, where the word shows up in Aran. The words are grouped by topic, and each group builds on the previous one, so read the groups in order.

A1. The data Aran protects

Term	Plain meaning	In Aran
PII (personally identifiable information)	Any data that can point to one real person: name, ID number, phone number, address.	Everything Aran looks for is PII.
Sensitive data	PII whose leak causes real harm: fraud, identity theft, fines.	Aadhaar, PAN, card, UPI ID, OTP, mobile, name, address.
Aadhaar	India's 12-digit national ID from UIDAI. The first digit is 2 to 9. The last digit is a **check digit** calculated with the **Verhoeff** method. Usually written 4-4-4, like `2345 6789 0124`.	The most important identifier. Detected exactly with Verhoeff.
PAN	Permanent Account Number from the Income Tax department. Ten characters: 5 letters, 4 digits, 1 letter, like `ABCPK1234L`. The 4th letter tells the holder type (P person, C company, H family, F firm, T trust). It has a check letter, but the method is not public.	Detected by its rigid shape plus context.
Card number (PAN in the card world, but we say "card" to avoid confusion)	A 13 to 19 digit payment card number, usually 16. The last digit is a **Luhn** check digit.	Detected exactly with Luhn plus the issuer prefix.
BIN / IIN	Bank or Issuer Identification Number: the first 6 or 8 digits of a card, which say which bank and network issued it. RuPay starts 60, 65, 81, 82 or 508; Visa 4; Mastercard 51 to 55 or 2221 to 2720.	Used as extra evidence that a 16-digit number is a real card. Also why "first 6 + last 4" masks leak more than people think.
UPI ID / VPA	Unified Payments Interface address (Virtual Payment Address), like `ravi@oksbi`. The part after `@` is a handle from a finite list published by NPCI.	Looks like an e-mail, so it needs a classifier.
OTP	One-Time Password: 4 to 8 random digits, valid for a few minutes. It has no checksum and no pattern.	The hardest thing to detect. Aran detects it because it was *issued*, not by how it looks.
Mobile number	Indian mobile: 10 digits starting with 6, 7, 8 or 9.	Detected by shape plus context words and, optionally, the bank's own customer list.
IFSC	Indian Financial System Code of a bank branch: 4 letters, a 0, then 6 characters.	A supporting identifier.

Term	Plain meaning	In Aran
Quasi-identifier	A field that is harmless alone but identifies someone in combination: date of birth, PIN code, gender. A famous study found 87% of Americans are unique by ZIP code, gender and birth date.	Why Aran measures what *combinations* of fields reveal (the leakage ledger).
KYC	Know Your Customer: the identity checks a bank must do, and the records it must keep by law.	KYC data has a legal hold, so it cannot be erased on request. Aran time-locks it.

A2. Check digits: the heart of Aran

Term	Plain meaning	In Aran
Check digit	One extra digit added to a number, calculated from the other digits, so that a typing mistake makes the whole number "invalid".	Aadhaar's last digit and a card's last digit are check digits.
Checksum	The calculation that produces or tests a check digit. A number "passes its checksum" when the check digit matches.	Real identifiers always pass. Aran's tokens always fail. This is the core idea.
Luhn algorithm (mod 10)	Invented by Hans Peter Luhn at IBM, 1960 patent. From the right, double every second digit, subtract 9 from any result over 9, add everything; the total must end in 0.	Card numbers.
Verhoeff algorithm	Invented by Jacobus Verhoeff, 1969. Uses a multiplication table from a mathematical structure called the **dihedral group D5** plus a permutation table. Stronger than Luhn: it catches every single wrong digit and every swap of two neighbouring digits.	Aadhaar. Code: `research/aran_core.py`, functions `verhoeff_valid` and `verhoeff_check_digit`.
Dihedral group D5	The 10 ways you can rotate or flip a regular pentagon and have it look the same. Combining two of these moves gives another move, and the order matters (rotate-then-flip is not flip-then-rotate). That "order matters" property is exactly why Verhoeff can notice swapped digits.	The 10×10 table `_D` at the top of `aran_core.py` is D5's multiplication table.
Group theory / modular arithmetic	The maths of structures like D5 (groups) and of "clock" arithmetic where you only keep the remainder (mod 10 means only the last digit matters).	The theory under Theorem 1.
Single-digit error, adjacent transposition	The two most common typing mistakes: one wrong digit (`1234` → `1284`), or two neighbours swapped (`1234` → `1324`).	Because Verhoeff always catches these, Aran can *repair* a typo by trying all 108 one-digit edits and 11 swaps of a 12-digit number.

Worked Luhn example. Find the check digit for the card body `7992739871`. Start from the rightmost digit and double every second digit: 1, 8, 3, 2 and 9 become 2, 16, 6, 4 and 18. Any result above 9 has 9 taken away, so 16 becomes 7 and 18 becomes 9. The doubled digits now add to 2 + 7 + 6 + 4 + 9 = 28; the untouched digits 7, 9, 7, 9, 7 add to 39. The total is 67, and the check digit is whatever brings it up to the next multiple of 10, which is 3. So `79927398713` passes Luhn, and changing any single digit makes it fail. Verhoeff works the same way in spirit but uses the D5 tables instead of doubling; Learn 2 walks through it digit by digit.

Worked Aran example (generated number, belongs to no one, demo key of all zeros, output of `research/aran_core.py`):

```
Real Aadhaar (passes Verhoeff):         234567890124   → valid
Token issued to the CRM:                249826846187   → fails Verhoeff
Token issued to the loan vendor:        098261586412   → fails Verhoeff
Token issued to the log platform:       019638092692   → fails Verhoeff
Each token decrypts back to:            234567890124
```

Same customer, three different tokens, one per destination, all 12 digits, none of them a valid Aadhaar.

A3. Data protection words

Term	Plain meaning	In Aran
Masking	Hiding part of a value for display, like `XXXX XXXX 0124`. Cannot be reversed.	Aran measures how much a mask still reveals.
Tokenisation	Replacing a sensitive value with a stand-in (a **token**) that has no meaning by itself. An authorised system can swap it back (**detokenise**).	Aran's main protection.
Token	The stand-in value. Aran's tokens keep the original shape (12 digits for Aadhaar) so the bank's software keeps working.	Checksum-complement tokens (M1).
Vault / vault mode	A protected table that maps token → real value. In vault mode the token is random, and only the vault links it to the real value.	Used for anything that must be erasable (M5).
Vaultless mode	The token is calculated by encryption from the real value, so no table is needed. Fast, but every customer's token can be reversed with one key, so it cannot be erased per customer.	Used only for short-lived flows.
Pseudonym	A consistent stand-in label, like `<PER:219779fa46>` for a person's name.	Names, PAN, UPI, mobile and address in log lines become keyed pseudonyms.
Placeholder	A typed stand-in inside an AI prompt, like `<AADHAAR_1>`.	The AI gateway.
Format-preserving	Protection that keeps the format (same length, digits stay digits).	FF1 tokens.

Term	Plain meaning	In Aran
Sink	Any destination data goes to: a log platform, a CRM, a vendor, an analytics warehouse, an AI provider.	Every sink gets its own tokens, so a leaked token names its sink (M3).
Source	The system that sends the data: the mobile app, core banking, the loan system.	The immune layer learns which source normally sends what to which sink.
Data flow	Data moving from a source to a sink.	Aran is a "data-flow firewall": it sits on these flows.
Choke point	A place all data must pass through, so one guard covers everything: the API gateway, the log shipper, the export path, the AI gateway.	Where Aran is installed.
Firewall	Normally a guard that blocks network connections. Aran is a firewall for *data*: it lets the traffic through but changes the sensitive values inside it.	The name of the whole system.
Data lineage	A record of where each piece of data came from and went.	Aran records each protect event as a flow.
Leak / breach / exfiltration	Sensitive data ending up where it should not be. Exfiltration is when someone takes it deliberately.	
Dump	A file of leaked records, often posted for sale on the dark web or Telegram.	Aran can look at a dump and say which sink leaked it.
Attribution	Finding who or what is responsible for a leak.	M3.
Erasure / right to erasure	A person's legal right to have their data deleted.	M5, crypto-shredding.
Retention / legal hold	A legal duty to *keep* some data for a time, even if the person asks for deletion (KYC records, security logs).	Why Aran's keys are per purpose, with a time-lock for KYC.
Threat model	A list of who might attack, how, and what they can do. Aran's: careless code, insiders, vendors, outside attackers.	Section C2.

A4. Cryptography

Term	Plain meaning	In Aran
Encryption	Scrambling data with a **key** so only someone with the key can read it.	Vault entries, FF1 tokens.
Key	A secret random number that controls encryption. Measured in bits; 256 bits means 2^256 possible keys.	One key per customer per purpose.
Symmetric encryption	The same key locks and unlocks. Fast.	AES.
AES	Advanced Encryption Standard, the world's standard symmetric cipher. AES-128 and AES-256 differ in key size.	Under FF1 and the vault. Upgraded to AES-256 for quantum safety.

Term	Plain meaning	In Aran
AES-GCM	AES in a mode that also detects tampering.	Encrypts each vault entry.
Block cipher	A cipher that scrambles fixed-size blocks (AES uses 16 bytes).	
Pseudorandom permutation (PRP)	A keyed shuffle: every input maps to exactly one output and back, and without the key the outputs look random.	A cipher is a PRP. FF1 is a PRP on 12-digit numbers.
FF1	A **format-preserving encryption** method standardised by NIST (SP 800-38G). Encrypts a string of digits into another string of the same length, using 10 rounds of a Feistel structure on top of AES.	Builds Aran's vaultless tokens. Tested against NIST's official sample vectors.
Feistel network	A way to build a cipher: split the input in two halves, scramble one half using the other and the key, swap, repeat. Always reversible.	FF1 uses 10 Feistel rounds. The animated tour shows all 10 rounds live.
Tweak	An extra public input to a cipher that changes the output without changing the key.	Aran uses the sink's name as the tweak, so each sink gets a different token.
Hash function	A one-way fingerprint of data: same input, same output; you cannot go backwards; a tiny change gives a completely different output.	SHA-256.
SHA-256	The standard 256-bit hash.	The Merkle log and the audit chain.
HMAC	A keyed hash: a fingerprint that only someone with the key can compute.	Registries store HMAC(key, Aadhaar) instead of the Aadhaar. Also picks each sink's wrong-digit offset.
Public-key cryptography	Two keys: a public one you share and a private one you keep. Used for signatures and for agreeing on keys over the internet.	Erasure receipts, key backups.
Digital signature	Proof, made with a private key, that a document came from you and was not changed. Anyone with the public key can check it.	Every erasure receipt is signed.
Ed25519	A fast, small, modern signature scheme (classical, not quantum-safe).	First receipts used it alone; now paired with ML-DSA.
Key exchange / X25519	A way for two computers to agree on a secret key over an open network. X25519 is the common modern method (classical, not quantum-safe).	Used when keys are backed up to another site; now paired with ML-KEM.
HSM	Hardware Security Module: a tamper-resistant box that holds keys and does crypto inside it, so keys never leave.	Where a real bank would keep Aran's keys.
KMS	Key Management Service: software (often cloud) that manages keys.	The prototype uses a local software KMS.

Term	Plain meaning	In Aran
Merkle tree	A tree of hashes: each leaf is the hash of a record, each parent is the hash of its two children, and the top is the **root**. Changing any record changes the root.	Every erasure is a leaf in Aran's Merkle log.
Inclusion proof	The short list of sibling hashes that proves a given record is in the tree, checkable by anyone with the root.	The erasure receipt contains one; the website re-checks it inside your browser.
Transparency log (RFC 6962, updated as RFC 9162)	An append-only Merkle log anyone can audit, made famous by Certificate Transparency for web certificates.	Aran's erasure log follows this standard.
Hash chain	A log where each entry includes the hash of the previous one, so editing an old entry breaks every later link.	The AI gateway's audit log.
Crypto-shredding	Deleting data by destroying the key it was encrypted with. Every copy, anywhere, becomes unreadable at once.	M5.
Perfect secrecy (Shannon, 1949)	A ciphertext reveals nothing at all about the message.	Proof of Theorem 3: once the key is gone, a random token carries zero information about the customer.
Receipt	Here: a signed, Merkle-logged proof that a specific key was destroyed at a specific time.	What the customer and the regulator get after erasure.

A5. Probability, detection and information

Term	Plain meaning	In Aran
True / false positive	A positive is "the detector said sensitive". It is *true* if it really was, *false* if not. A false positive is also called a **false alarm**.	Plain regex raised 67,481 OTP false alarms on 32,000 real log lines.
False negative / miss	The detector said "fine" but it was sensitive.	
Precision	Of everything flagged, how much was really sensitive.	
Recall	Of everything sensitive, how much was flagged.	"Mobiles: 68% recall" means 32% were missed.
F1 score	One number combining precision and recall (their harmonic mean). 1.0 is perfect.	NER results.
AUROC	How well a score *ranks* real above fake, from 0.5 (coin toss) to 1.0 (perfect), regardless of the threshold.	UPI classifier AUROC 1.00.
Threshold	The score above which you say "sensitive".	Each sink can have its own threshold.

Term	Plain meaning	In Aran
Probability, prior, posterior	The prior is how likely something is before looking at evidence; the posterior is after.	
Bayes' rule	How to update a probability when you see evidence.	The detector combines evidence with it.
Likelihood ratio	How much more likely a piece of evidence is if the value is real than if it is noise. Multiply the odds by it. A checksum pass has ratio 10 (real numbers always pass; random numbers pass 1 time in 10).	Each clue is a likelihood ratio.
Neyman-Pearson lemma	A theorem: thresholding the likelihood ratio is the best possible test for any chosen false-alarm rate.	Lets the bank choose the false-alarm rate it can afford.
Calibration	A detector is calibrated when "90% sure" is right 90% of the time.	Aran's detector is calibrated so its scores mean something.
ECE (expected calibration error)	Average gap between stated confidence and actual accuracy. Lower is better.	0.015 at first, 0.0065 later.
Isotonic regression	A simple method that bends raw scores into calibrated probabilities.	Used in the detector.
Bloom filter	A compact bit array that answers "have I seen this before?" with a small, known chance of false "yes" and no false "no". Cannot delete.	First design for the customer registry.
Cuckoo filter	Like a Bloom filter but supports deletion (Fan et al., 2014).	The built registry, because erased customers must be removable.
Birthday bound	The surprising chance that two random items collide (23 people: 50% share a birthday).	Bounds the chance two sinks' tokens collide (Theorem 2).
Bit (of information)	One yes/no answer. N equally likely possibilities = log2(N) bits. 1,000 candidates ≈ 10 bits.	How Aran measures what a mask leaks.
Entropy / min-entropy	Entropy measures uncertainty. Min-entropy measures how hard the *best single guess* is: log2 of the number of equally likely candidates left.	The leakage ledger keeps every customer above a minimum (for example 20 bits ≈ 1 million candidates).
Quantitative information flow (QIF)	The field that measures in bits how much a system leaks (Smith, 2009).	Theory behind M4.
Dynamic programming	Solving a counting problem by building up from smaller answers.	Counts exactly how many valid Aadhaar or card numbers fit a mask.

A6. AI, machine learning and NLP

Term	Plain meaning	In Aran
AI / machine learning (ML)	Programs that learn patterns from examples instead of being given rules.	Names, addresses, OTP and UPI context, calibration.

Term	Plain meaning	In Aran
Model	The learned thing that makes predictions.	`app/server/models/detector.pkl`.
Training / test data	Training data teaches the model; test data, kept separate, checks it.	Aran trains on 8 log systems and tests on 8 others it never saw.
Feature	One measurable clue the model looks at (for example "the word before is *mobile*").	
Overfitting	Learning the training examples by heart and failing on new ones.	Why templates and names are split into train and test halves that share nothing.
NLP	Natural Language Processing: AI for text.	
NER	Named Entity Recognition: finding names, places, organisations and addresses in text.	Names and addresses have no checksum, so they need NER.
CRF (conditional random field)	A classic, fast, CPU-only model that labels each word in a sentence, looking at its neighbours.	Aran's NER. Trains in about 5 minutes on a laptop, no GPU needed.
Character n-grams	Slices of 2, 3, 4 letters (for "otp": "ot", "tp", "otp"). Work across English, Hindi and Hinglish without a separate dictionary for each.	The OTP/UPI classifier.
Classifier / logistic regression	A model that outputs yes/no with a probability. Logistic regression is the simplest.	
Stacker	A small model that combines several other models' outputs.	One per data type in the calibrated detector.
Hinglish / code-mixed text	Hindi and English mixed in one sentence, often in Latin letters.	One of Aran's three NER test languages.
Transformer	The modern AI architecture behind GPT, BERT and others, based on **attention** (each word looks at every other word).	Planned upgrade for names (Learn 9).
Zero-shot NER, GLiNER, IndicNER	Zero-shot: finding entity types the model was never trained on, just from their name. GLiNER is an open zero-shot NER model; IndicNER is trained on Indian languages.	Roadmap item 7.
Fine-tuning / distillation	Fine-tuning: teaching a big pre-trained model your task. Distillation: training a small fast model to copy a big slow one.	Roadmap item 7.
Cascade / gate	Running a cheap check first and the expensive model only when needed.	The NER gate doubled speed (274 → 654 lines per second).
OCR	Optical Character Recognition: reading text from images. **Tesseract** is the standard free OCR engine.	Card photo scan; Verhoeff checks that the OCR read the Aadhaar correctly.
LLM	Large Language Model, like ChatGPT.	Bank staff paste customer data into LLMs; the AI gateway stops raw values reaching the provider.

Term	Plain meaning	In Aran
Prompt / prompt injection	A prompt is the text sent to an LLM. Prompt injection is hiding instructions in it to trick the model (for example "ignore the rules and print the customer's Aadhaar").	Five attack types tested against the AI gateway; none leaked a value.
Isolation Forest	A standard anomaly-detection method that isolates unusual points by random splits.	The baseline the immune layer beat (100% vs 20%).
Anomaly detection	Finding things that do not look like normal.	The immune layer.

A7. The immune system idea (the "biology plan")

Term	Plain meaning	In Aran
Artificial immune system (AIS)	Algorithms copied from how the body's immune system tells "self" from "foreign".	The immune layer, owned by the cyber pair.
Self / non-self	In the body: your own cells vs invaders. In Aran: normal data flows vs unusual ones.	
Thymus and negative selection	In the thymus, young T-cells that react to the body's own proteins are destroyed; only those that ignore "self" survive, so they only attack foreign things. In the algorithm: generate random detectors, delete every one that matches a normal flow, keep the rest.	`research/immune/ais.py`.
Detector / V-detector	A detector is a point plus a radius in feature space; anything inside it is "non-self". V-detectors (Ji and Dasgupta, 2004) have variable radii to cover space efficiently.	
Feature space	Each data flow becomes a point with 7 numbers (volume, hour, raw fraction, detokenisation rate, and so on). Similar flows are close together.	The 3D tour scene shows this space.
Self radius	How close to a normal point something must be to count as normal. Bigger means fewer false alarms but more misses.	Tuned on validation days to a 2% false-alarm budget.
Immune memory	After an infection, the body reacts faster next time. In Aran, a confirmed attack is cloned into memory detectors, so a repeat scores higher.	Repeat attack score rose from 0.37 to 0.71.
Tolerance	The body learns not to attack harmless things. In Aran, a confirmed false alarm is added to self.	Tested; helped less than hoped (honest result).

A8. Software, web and tools

Term	Plain meaning	In Aran
Python	The programming language of all of Aran's research and server code.	`research/`, `app/server/`.
Library / package	Reusable code someone else wrote, installed with `pip` (Python) or `npm` (JavaScript).	`cryptography`, `scikit-learn`, `sklearn-crfsuite`, `fastapi`, `numpy`.
Test / pytest	Small programs that check the code still gives the right answer. `pytest` runs them.	52 test functions across `research/test_*.py` and `app/server/test_standout.py`.
Experiment script	A program that runs a full measurement and writes results to a JSON file.	`experiment*.py` → `results*.json`.
JSON	A plain-text format for structured data: `{"name": "Ravi", "age": 30}`.	API messages, results files.
Git, repository (repo)	Git tracks every version of every file. A repo is a project folder under git.	`github.com/itzz-inbathamizhans/aran`, local copy `D:\Aran`.
Commit, branch, merge, push, pull	A commit is a saved snapshot. A branch is a separate line of work. Merge joins branches. Push sends commits to GitHub; pull fetches them. `main` is the main branch.	Every piece of work was merged into `main`.
API	Application Programming Interface: a defined way for programs to talk to each other.	Aran's server has about 50 API **endpoints**, like `POST /api/process`.
HTTP, endpoint, GET, POST	HTTP is the language of the web. An endpoint is one URL that does one job. GET reads, POST sends.	
Server / backend	The program that does the real work and answers requests.	FastAPI app in `app/server/`.
Frontend	What runs in the browser and what people see.	`app/ui/` (main) and `app/web/` (classic).
FastAPI, uvicorn	FastAPI is a Python library for building APIs; uvicorn is the program that runs it.	`uvicorn main:app --port 8000`.
localhost	"This computer". `http://localhost:8000` only opens on the machine running the server.	Why the site opened on the laptop but not on a phone at first.
Proxy / reverse proxy / gateway	A program that sits between a client and a server, passing traffic and able to change it.	Aran's place in a real bank.
HTML, CSS, JavaScript	HTML is the page structure, CSS the look, JavaScript the behaviour.	
React, Vite	React builds interfaces from components; Vite builds and serves React apps.	The classic console at `/classic`.

Term	Plain meaning	In Aran
three.js, WebGL	WebGL draws 3D graphics in the browser; three.js makes it easy.	3D firewall scene, 3D immune space, the hold-to-blast hero.
GSAP, Lenis	GSAP is an animation library; Lenis gives smooth scrolling.	The team's design.
SVG	Vector drawings in a web page that stay sharp at any size.	The animated tour scenes.
Design tokens	Named design values (`{primary-color}` , spacing, fonts) used instead of fixed colours, so a design can be re-themed in one place.	The UI design spec and `app/ui/css/tokens.css` .
i18n	Internationalisation: making a site work in several languages.	English, Tamil and Hindi switch.
Service worker, localStorage	Browser features that let a page cache answers and keep small data, so it works offline.	Presenter mode's offline replay.
QR code, Web Speech	QR code: a square barcode phones can scan. Web Speech: the browser's speech-to-text.	The phone arena.
Docker	Packs an app and everything it needs into one container that runs the same everywhere.	`Dockerfile` .
Render	A cloud hosting service.	Planned public hosting; blocked because the repo is private and Render had no access.
Obsidian	A note-taking app that works on a folder of Markdown files (a "vault") with links between notes.	`D:\Aran` is also an Obsidian vault: `Home.md` , `notes/` .
Notion	An online workspace.	The "Second Brain" project page and handoffs.
Markdown	Plain text with simple marks (`#` heading, `**bold**`), readable as is and convertible to web pages or PDF.	Every doc in this repo.

A9. Law and regulation

Term	Plain meaning	In Aran
DPDP Act 2023	India's Digital Personal Data Protection Act. Penalties up to ₹250 crore for failing reasonable security safeguards, ₹200 crore for failing to report a breach.	The main law Aran helps banks meet.
DPDP Rules 2025	The detailed rules, notified 13 November 2025. Core duties apply from 13 May 2027. Breach report within 72 hours; logs kept at least 1 year.	Erasure must coexist with log retention.

Term	Plain meaning	In Aran
Data Fiduciary / Data Principal / Data Protection Board	The organisation holding data (the bank) / the person the data is about (the customer) / the regulator that enforces DPDP.	
CERT-In Directions 2022	India's cyber-emergency team rules: report listed incidents within 6 hours; keep ICT logs for 180 days in India.	Logs must be kept, so they must be safe to keep.
UIDAI Aadhaar Data Vault	UIDAI rule (Circular 8 of 2025): Aadhaar numbers may be stored only in a vault with an HSM and AES-256; other systems use reference keys.	Aran's tokens extend this to every sink.
RBI	Reserve Bank of India: card tokenisation and payment-data rules; FREE-AI framework (August 2025) for governing data going into AI.	The AI gateway.
PCI DSS	The card industry's security standard. "First 6 and last 4" digits is the most anyone may show.	Aran measured that this still leaves only 100,000 candidates.
DST quantum-safe task force (Feb 2026)	Government of India report: banking fully migrated to quantum-safe crypto by 2033; vendors to provide cryptographic inventories (CBOMs) from 2027 to 2028.	The post-quantum work.

A10. Quantum computing and post-quantum cryptography

Term	Plain meaning	In Aran
Quantum computer / CRQC	A computer using quantum effects. A *cryptographically relevant* quantum computer (CRQC) is one big enough to break today's public-key crypto. None exists yet.	
Shor's algorithm	A quantum algorithm that breaks RSA, X25519, Ed25519 and all similar public-key methods completely.	Breaks Aran's original receipts and key backups.
Grover's algorithm	A quantum search that halves the strength of symmetric keys (AES-128 → 64-bit strength; AES-256 → 128-bit).	Why Aran moved FF1 to AES-256.
Harvest now, decrypt later	An attacker records encrypted traffic today and decrypts it once a quantum computer exists.	Leads to Aran's finding, "retroactive un-erasure".
PQC (post-quantum cryptography)	New public-key methods believed safe against quantum computers.	
ML-KEM (FIPS 203), ML-DSA (FIPS 204), SLH-DSA (FIPS 205)	NIST's 2024 post-quantum standards: ML-KEM for key exchange, ML-DSA for signatures, SLH-DSA for very conservative hash-based signatures.	Aran uses ML-KEM-768 and ML-DSA-65.

Term	Plain meaning	In Aran
Hybrid	Using a classical and a post-quantum method together, so the result is safe if *either* one holds.	X25519 + ML-KEM-768; Ed25519 + ML-DSA-65.
Downgrade attack	Stripping the stronger half so the system falls back to the weaker one.	Blocked: the algorithm name is inside the signed bytes.
Mosca's inequality	If (how long data must stay secret) + (how long migration takes) > (years until a quantum computer), you are already late.	Aran's priority table: Aadhaar and PAN first.
CBOM	Cryptographic Bill of Materials: a list of every crypto algorithm a system uses (CycloneDX format).	`python -m pqc.cbom` writes one.

A11. Research words

Term	Plain meaning	In Aran
Problem analysis	Working out what the problem really is, beyond the wording.	Aran's reframing: a bank cannot *prove* where each identifier is.
Baseline	The obvious solution most teams would build, used as the yardstick.	Detect-then-redact proxy with regex.
Literature survey	Reading and summarising existing research, rules and incidents.	About 40 sources in the research document.
Research gap	Something existing work does not solve.	G1 to G6.
Novelty	What is genuinely new.	Section C6.
Theorem / proof sketch	A statement proven true by logic, and a short version of the proof.	Theorems 1, 2 and 3.
Experiment / dataset	A planned measurement / the data it runs on.	E1 to E19; Loghub, CoNLL-2003, HiNER and others.
Negative result	A test where the idea did *not* work, reported honestly.	Immune layer on raw BGL log lines.
Reproducibility	Anyone can re-run the code and get the same numbers.	Every experiment has a script and a results file.
Red team	Attacking your own system on purpose to find holes.	E15: 26 hiding tricks.
Critical review	Attacking your own design on paper before building it.	11 weaknesses found and fixed in the design.

Part B. The story, from zero

All times are Indian Standard Time (IST). Everything so far happened over about 43 hours, from just after midnight on 16 October to the evening of 17 October 2026, with this document written on 19 October.

B1. The starting point

The team. Four students at KARE (Kalasalingam Academy of Research and Education): Inbathamizhan S (lead, cybersecurity), Aasif Ahmed A (cybersecurity), SN Arun Vijay (AI/ML) and Bharadhwaj M (AI/ML).

The problem statement (domain: Banking, Financial Services and FinTech), given as an image and copied into the first README:

> *Prevent Sensitive Data Leakage.* Financial institutions handle large volumes of sensitive customer information across APIs, applications and system logs. Account numbers, card details, identity documents and personal contact information pass through many systems every day, and any of them can end up exposed in a request payload, a debug log or a shared report. **Challenge:** create a solution that can identify sensitive customer information in API traffic and application logs and automatically protect or mask it before it is stored or shared. **Technologies:** AI / NLP, Data Privacy, APIs, Cybersecurity, Cryptography.

The idea you brought to it (your own words, cleaned up): an AI-powered data-flow firewall for banks that detects Aadhaar, PAN, card numbers, UPI IDs and OTPs in APIs, logs and AI tools; automatically masks or encrypts them before they are stored or shared; tracks where data travels; pinpoints the source of leaks; and supports DPDP-compliant erasure. You asked for real-world data (nothing fake), a problem analysis, a literature survey, a search for open research problems, a novel plan grounded in maths or science, and a critical review, and you asked for a solution that is "unique and strong".

The name. *Aran* (அரண்) is Tamil for *fortress* or *protective wall*.

B2. Timeline at a glance

When (IST)	What happened	Result in the repo
16 Oct, 00:23	Project opened. First request: "create a repo named Aran".	GitHub repo `itzz-inbathamizhans/aran`
00:29	You asked to clone it to `D:\Aran` on your laptop (named Zeran), work locally, commit only as "Inbathamizhan S" with no AI words, and make one first commit with the problem statement.	Commit `93f1897`: README with the problem statement
00:34 to 00:42	Working rules set: save every session to Notion ("Second Brain"), keep an Obsidian vault in `D:\Aran`, push to GitHub only on "save".	Rules in project instructions and `notes/Workflow.md`

When (IST)	What happened	Result in the repo
00:38	**The research request.** Analyse, survey, find gaps, design a novel maths-based solution, review it.	Research thread starts
00:43	You approved the 6-step research plan.	
00:57	Research document finished; prototype code and first experiments (E1 to E4) on real logs.	Research document; `research/aran_core.py`, `experiment.py`
01:01	You asked: "does the project use AI?" Answer: partly; the core is maths on purpose, AI is for names, addresses, OTP/UPI context and calibration, and those parts were not built yet.	
01:03	You asked to add the AI parts and the "biology plan" too, and gave the team split (cyber pair, AI/ML pair).	AI + immune plan, split by pair
01:05	**New standing rule:** before any plan, write beginner learning docs, one per topic, because the team is new to this technology.	Learn 1 to 7 written by 01:11
01:08	You moved the paused build to its own thread.	"Build AI and immune layers" thread
01:12	**New standing rule:** every artifact and update is saved in the repo, linked from the README, and pushed.	Commits `0d48481`, `4092e09`
01:15	You asked for a design-system prompt for the website, to use in a design tool.	`artifacts/website/Aran-Website-Design-System-Prompt.md`
01:27	You asked to rewrite Learn 1 to 7 with much more explanation and examples.	Commit `1803932` (expanded docs)
01:35 to 03:15	Work paused, then resumed on its own.	
03:51	AI and immune layers finished and measured (E5 to E9).	Commit `f8175a5`; AI and immune layers report
04:02 to 04:04	You asked "is the project done?", "what is the project?", "how does it work?"	
04:04	You asked what more could be researched and built.	Roadmap (`4fea87d`), Learn 8 to 10 (`69872f1`)
04:31	"Yes, start": the roadmap's first items.	Normaliser and registries (`3944f75`, E10 to E12)
16 Oct, 16:45	You asked whether GitHub, Obsidian and Notion were all in sync.	Vault committed (`98dca8e`)
16:49	"Is the project completely ready?" Honest answer: no, erasure was only designed, and there was no live demo.	

When (IST)	What happened	Result in the repo
16:51	You asked how to build a frontend with a design tool, and to continue the remaining work in parallel.	Frontend guide (`1187ba5`); "continue all" starts the ledger/gateway/red-team thread
16:54	"I need a real site, no simulated values."	
17:27	**Live console built**: FastAPI server on the real firewall, React frontend, crypto-shredding with Merkle receipts (roadmap item 4).	Commit `02b11d6`; Learn 11
17:49	"Run the site." Render could not open the private repo, so it was started on your laptop at `http://localhost:8000` from a separate copy, `D:\Aran-live`.	
17:58	You asked for a frontend that explains everything to reviewers who know nothing.	Guided tour, 12 chapters (`afcebef`)
18:09	In the parallel thread: leakage ledger, AI gateway, red team and speed-ups.	Commit `881dd0e` (E13 to E16); Learn 12
18:19 to 18:22	You asked "how does it really work", "in real life", "is it a product or a backend?"	Answers in Part H
18:22	"Fewer words, more visuals, animations like hashing."	Animated tour (`cc579fb`); Learn 13
18:31	You asked to research and implement post-quantum cryptography.	Commit `9a17c11` (E17 to E19); Learn 14
20:10	"Add 3D, more motion, easier to understand."	3D scenes (`5f8ffd6`); Learn 16
20:12	You asked for a full page-by-page design spec with colours only as placeholders, for your own redesign.	UI design spec (`fe07eda`), design references (`9b588d9`), three mockups (`7e61065`); Learn 15
21:41	Your own finished design in `D:\Aran\ui-design` was to become the real site, wired to the backend.	Copied from your laptop; wired by 00:19 (`66b17d7`); Learn 17
17 Oct, 00:39	"Is the backend really replacing every simulated value?" An audit found two places still quoting fixed numbers; both fixed.	Commit `14bbe6d`
01:06	"What can be added so the site stands out?"	Ideas (`df6abcb`), then 8 demo features (`592102e`); Learn 18 to 20
17 Oct, daytime	Frontend audit fixes on phones and tables.	`a7a2a44`, `0343c41`
18:56	"Fix all, commit to main, pull on main." Your local work in `D:\Aran` (design prototype, site research, the Iravora Void site, refreshed screenshots) was committed and merged.	`57a3b82`, merge `1554442` (today's `main`)
19 Oct	This document.	`artifacts/Project Story/`

B3. Setting up how we work (16 Oct, 00:23 to 00:45)

Before any research, you set the working rules that every later step followed. They matter because they explain why the repo looks the way it does.

1. **Local first.** The real working copy is `D:\Aran` on your laptop. GitHub gets changes when you say *save* or *push*.
2. **Your name only.** Every commit is authored "Inbathamizhan S", with no tool credits anywhere in the repo.
3. **Everything is recorded.** `D:\Aran` is also an Obsidian vault: a note per thread in `notes/threads/`, a daily log in `notes/daily/`, and indexes of artifacts and branches. Sessions were also saved to a Notion "Second Brain" project page.
4. **Plan before big tasks.** Each large task starts with a short step plan and waits for your OK.
5. **Learn first.** From 01:05 onward: before any new plan, the team gets beginner learning docs, one per topic.
6. **Every artifact goes in the repo** under `artifacts/`, with Markdown and PDF copies, linked from the README, and pushed.

The first commit (`93f1897`) contained only the README with the problem statement, so you could check it before anything else went in.

B4. The research (16 Oct, 00:38 to 00:57)

You asked for analysis, survey, gaps, a novel solution derived from maths or science, and a review. A six-step plan was proposed and you approved it:

Step	What
1	Analyse the problem statement and give a baseline solution
2	Literature survey from real sources: DPDP Act and Rules, RBI and CERT-In rules, real Indian bank breaches, PII detection, taint tracking, format-preserving encryption, crypto-shredding
3	Find the research gaps current tools leave open
4	Design a novel solution grounded in maths and science
5	Critically review it, attack it, and fix the weak points
6	Write it up as a shareable document

One rule was set at the start: **"real data" means real public facts and real public datasets, never a real person's Aadhaar, PAN or card number**, because using one would itself be a data leak. Every test identifier in the project is generated with a valid checksum and belongs to no one.

The experiment that measured the gaps ran on **Loghub**, a public collection of real production logs from 16 systems (HDFS, Hadoop, Spark, OpenStack, Linux, Windows, Mac, Android, OpenSSH, Apache and others), 32,000 lines in total. These logs contain no Indian identifiers, so **every hit on them is a false alarm**, which makes them a clean test of how noisy a detector is.

The result was the research document *Aran: Novel Data-Flow Firewall for Indian Banks*, with five mechanisms (M1 to M5), three theorems, eleven review fixes and four experiments. Part C explains exactly how this was done and how the solution was derived.

B5. "Does it use AI?" and the team split (01:01 to 01:05)

The first prototype had no AI model at all: it ran on checksums, encryption and statistics. That was deliberate, because a checksum is exact and a model would only add errors where exactness is possible. But names and addresses have no checksum, OTPs have no structure, and the problem statement lists AI/NLP. So the plan added:

Part	Owner pair
NER for names and addresses (English, Hindi, Hinglish)	Bharadhwaj M, SN Arun Vijay (AI/ML)
OTP and UPI context classifier, plus the calibrated detector	Bharadhwaj M, SN Arun Vijay (AI/ML)
Immune-system flow anomaly detector (your "biology plan")	Inbathamizhan S, Aasif Ahmed A (cybersecurity)
Wiring everything into the firewall and testing on real logs	Inbathamizhan S, Aasif Ahmed A (cybersecurity)

"Owner" in this project means the person responsible for understanding, presenting and maintaining that part. The research document had no biology section, so "biology plan" was read as an artificial immune system that learns normal data flows and flags unusual ones.

B6. Learning docs first (01:05 to 03:17)

Because the team is new to this technology, you made a standing rule: no plan without beginner docs first. The build paused and seven docs were written: Learn 1 Python and Tools, 2 Indian Identifiers and Check Digits, 3 Cryptography Basics, 4 Probability for Detection, 5 AI and NLP, 6 Artificial Immune Systems, 7 Data-Protection Law. At 01:27 you asked for much more explanation and examples, and all seven were rewritten with worked examples taken from Aran's own code and data, practice tasks with solutions, and free further reading. Every later plan got its own docs; there are now 20 (Appendix B).

B7. Building the AI and immune layers (01:08 to 03:51)

This ran in its own thread. It found real public datasets (CoNLL-2003 English news, HiNER Hindi, Hinglish NER tweets, the UCI SMS corpus, labelled BGL supercomputer logs), trained a CRF name-and-address finder, a character n-gram OTP/UPI classifier, a calibrated detector that fuses all the evidence, and the negative-selection immune layer; then wired them into one firewall and ran it on 16,000 real log lines from 8 systems the models had never seen (E5 to E9). The work paused once with the code written and 13 tests passing, then resumed and finished.

Headline: the firewall removed **83.5%** of injected sensitive values against **46.1%** for plain regex, while wrongly changing only **0.8%** of clean real lines (regex flagged 15.8%). The immune layer caught **100%** of six simulated attack types at 0.6% false alarms. It also reported its weak spots plainly: mobiles with no cue word (68%), English OTPs next to transaction words (80%), and a negative result for the immune layer on raw log lines.

B8. What next? The roadmap (04:04 to 04:26)

You asked what more could be researched and implemented. The roadmap (`docs/research/aran-roadmap.md`) listed 13 items in three tiers, each with owners, effort and the learning docs to read first. Tier 1 (needed before any demo): 1 live gateway and dashboard, 2 registries, 3 fixing the ML weak spots, 4 crypto-shredding with receipts, 5 normaliser and typo repair. Tier 2 (stand-out): 6 AI gateway, 7 transformer NER, 8 Tamil support, 9 red team. Tier 3 (after the hackathon): 10 OCR on KYC images, 11 immune layer on log templates, 12 speed, 13 paper and legal review. Item 14 (post-quantum) was added later. Learn 8 (Web APIs and Proxies), 9 (Transformers and Zero-Shot NER) and 10 (OCR) were written for these items.

B9. Normaliser, registries and typo repair (04:31 to 05:26)

You said “yes, start”, and roadmap items 5 and 2 were built first because they make every later number better. The **normaliser** undoes 12 ways of hiding a number (Devanagari, Tamil or full-width digits, URL, JSON and HTML escapes, dots, underscores, spaces, zero-width characters, split fields, base64). The **own-customer registry** keeps keyed fingerprints of the bank’s own identifiers in a cuckoo filter; the **trace-bound OTP registry** remembers each issued OTP together with the request it belongs to; **typo repair** finds a real Aadhaar or card number with one wrong digit. Results (E10 to E12): values removed rose from 83.5% to **89.7%**; mobiles and OTPs reached **100%**; hidden identifiers went from 8.3% to **100%**; one-typo identifiers from 0% to **99.2%**; no new false alarms.

B10. “Is it ready?” and the live console (16 Oct, 16:45 to 17:56)

In the afternoon you asked whether everything was synced, then whether the project was completely ready. The honest answer was no: erasure (M5) was only designed, and there was nothing a judge could click. You asked how to build a frontend using a design tool, and a frontend guide was written with two prompts (a design system and an 8-screen dashboard) and the list of API endpoints the dashboard would need.

Then you set a hard requirement: **a real site, with no simulated or duplicated values; everything must run for real.** The console was built in one step:

- a **FastAPI server** (`app/server/engine.py`, `main.py`) that runs the real firewall, detector, normaliser, registries and immune layer;
- **purpose-bound crypto-shredding** (`research/firewall/shredder.py`): one key per customer per purpose, random vault tokens, key destruction, an RFC 6962 Merkle log and a signed receipt, with a KYC retention lock (roadmap item 4);
- a **React frontend** with seven pages;
- a `Dockerfile` for hosting.

Only the customers’ identifiers are generated, because real ones must never be used. Hosting on Render needed Render to be given access to the private repo, which had not been done, so the site was started on your laptop instead, at `http://localhost:8000`, from a separate copy of `main` at `D:\Aran-live` (your `D:\Aran` had uncommitted edits, so it was left untouched).

B11. Making it understandable: the tour, animation and 3D (17:58 to 20:25)

You asked for the site to explain everything to reviewers and senior people who know nothing about the topic, by letting them use it. A **guided tour** of 12 plain-language chapters was added as the opening page, each with a "Try it" button running the real code and an "Under the hood" box for technical reviewers.

You then asked for fewer words and much more visual explanation, with animations "like hashing". The tour was rebuilt as **13 animated chapters**, each a headline, one line of text and a moving picture of a real server response: packets flowing through the firewall and coming out as tokens; 300 real log lines lighting up as false alarms for plain filters but not for Aran; the Verhoeff and Luhn maths running digit by digit; a token being built through the 10 real FF1 rounds; a probability gauge; 12 disguise cards flipping to "caught"; a leaked file flying to the dark web and a trace beam naming its source; the purpose key shattering on erasure with your browser re-checking the Merkle receipt; the immune radar; and a new chapter on AI chatbots.

You asked for 3D and more motion. Two **three.js scenes** were added, both driven by live data: a 3D firewall that real requests fly through, and the immune layer's real 7-dimensional feature space shown in 3D. Phones get a 2D map instead. Learn 13 and Learn 16 explain how the animations and 3D scenes are built.

During this period the site sometimes did not appear for you. Each time the server was checked from the laptop itself and was running; the causes were the browser showing an old cached copy (fixed with Ctrl+F5) and `localhost` only working on the laptop, not on a phone.

B12. Ledger, AI gateway, red team, speed (in parallel, 16:56 to 18:09)

While the console was being built, "continue all" started a parallel thread on the roadmap items not covered by the console: item 3 (fix mobile and OTP weak spots), item 6 (AI gateway for LLM prompts), item 9 (red team), the M4 leakage ledger from the research document, and a first step on items 7 and 12 (NER speed). Results (E13 to E16): values removed with no registries **91.0%**; mobiles and OTPs **100%**; customers a vendor could narrow below 20 bits by joining its systems **14,175 → 0**; raw values reaching an LLM through the gateway **100% → 7.8%** (all of them names or addresses; every number type 0%); hiding tricks fully caught **14 → 22 of 26**; two real detector bugs found and fixed; speed **274 → 654** lines per second. Two results went the wrong way and were reported as such (Part E).

B13. Post-quantum cryptography (18:31 to 20:05)

You asked how to use post-quantum cryptography to make Aran stronger. The research found that most of Aran is already quantum-safe, because its core is symmetric (AES, HMAC, SHA-256), and that three parts were not. It also produced a new finding: **crypto-shredding can be undone later** if a destroyed key ever travelled under quantum-breakable key exchange, because an attacker who recorded that traffic can recover the key once a quantum computer exists. The fixes: hybrid Ed25519 + ML-DSA-65 erasure receipts with downgrade protection, hybrid X25519 + ML-KEM-768 sealing for any key that leaves the vault, AES-256 for FF1, and a CBOM scanner (E17 to E19, Learn 14).

B14. Your own design becomes the real site (20:12 to 17 Oct, 00:59)

You said you would design a new UI yourself and asked for a complete list of every page and everything on it, with colours only as placeholders like `{input-color}`. The **UI design spec** covers design tokens, the app shell, 33 components, the 13 tour chapters and 8 pages with content, data bindings, animations and states, all API endpoints, responsive and accessibility rules, and a handoff checklist. A companion **design references** document picked real, human-designed websites for each page, with a checklist for avoiding generic "AI-looking" design, and three Overview mockups (calm dark, bank light, live data) showed possible directions. Learn 15 explains design specs and tokens.

You then built your own design in `D:\Aran\ui-design`: a Trionn-style dark 3D look with a hold-to-blast WebGL hero, smooth scrolling, word reveals and optional sound, written in plain HTML, CSS and JavaScript. Your notes in `site-research/` record the five reference sites studied for it (Percy Jackson template, Artemii Lebedev, dmcg, Trionn, Vlad Prozapas). Your design was copied from the laptop, and every element was **wired to the live server**: every number, token, receipt and alert now comes from a real API call. The earlier React console stayed at `/classic`. Learn 17 explains the wiring.

You then asked whether the backend had really replaced every simulated value. An audit found two places that still quoted fixed numbers (three "Under the hood" figures had gone out of date, and one sample mobile was made in the browser). Both now come from the server, and the stale figures were corrected from the result files.

B15. Standing out (17 Oct, 01:06 to 18:59)

You asked what could make the frontend stand out, since every team's site will look different but similar. An ideas list ranked eight features, top three first, and all of them were built (Learn 18 to 20 explain them):

- **Beat the firewall** (`#arena-live`, `#arena`): judges scan a QR code and try, from their own phones, to type, paste or speak a hidden Aadhaar, card or mobile past the live firewall, scored honestly on a live board.
- **Without Aran / with Aran** (`#versus`): the same real requests side by side, then a breach: raw identifiers and the DPDP penalty on one side, tokens and a trace naming the leaking sink on the other.
- **Leakage ledger** (`#ledger`): first 6 digits to one system and last 4 to another leave 100,000 candidates, and the ledger refuses the second disclosure.
- **Card photo scan** (`#scan`): Tesseract reads a sample ID card image and the identifiers are covered with tokens on the image.
- **Post-quantum** (`#quantum`): a simulated quantum attack opens a classically wrapped key backup but not the hybrid one.
- **My data** (`#me`): the customer's own phone view of where their data went, with erasure and a receipt checked on the phone.
- **Tamil and Hindi** language switch.
- **Presenter mode** (`?present=1`): a 3-minute path with clicker keys and an offline replay of recorded server answers in case the server fails.

On 17 October, audit fixes made the phone layout, tables and language picker work properly, and on your instruction all your uncommitted local work in `D:\Aran` was committed under your name and merged into `main` (`1554442`). Tests passed: server 8 of 8, research 45 of 45, and English, Tamil and Hindi each

complete at 107 of 107 lines. Three things were left off GitHub on purpose: the raw scraped text of other people's websites (`site-research/raw/`), a laptop-only settings file, and 24 local screenshots that differed from `main` (moved to `D:\Aran-local-backup-2026-10-17`, nothing deleted).

Side project in the repo. `iravora-void/` is a separate small website (a black-tea brand page where one 3D scene changes as you scroll), committed from your local folder. It is not part of Aran. Judging from its code, it looks like practice for the same scroll-driven 3D techniques the design uses (this is an inference; it has no notes of its own).

Part C. The research: papers, gaps and how the solution was derived

This part answers: what research was done, how it was done, which papers and rules were used, what each one gave Aran, and how the solution was derived from them. The full text is the research document; this is the guided version.

C1. Method: how the research was done

The research followed the six steps in B4, in this order, in about twenty minutes of focused work on the night of 16 October, and was extended by later threads:

1. **Reframe the problem.** Read the problem statement for what is actually missing, not for what it literally asks.
2. **Write the obvious solution first** (the baseline), so there is something concrete to beat.
3. **Survey three kinds of sources**: the rules a bank must follow, real leaks and what each teaches, and research papers grouped into lines of work.
4. **Measure the gaps on real data**, rather than only arguing them: the Loghub experiment (E1 to E4).
5. **Design from theory**: for each gap, find a piece of maths that closes it, and state what can be proven.
6. **Attack the design** (critical review), fix what breaks, and write down what is honestly new and what is not.

Later threads repeated the same method on each new piece: the AI and immune report, the normaliser report, the ledger/gateway/red-team report and the post-quantum report each have data, experiments, honest limits and sources.

C2. Step 1: reframing the problem

The problem statement says “identify sensitive data and mask it before it is stored or shared”. The research document’s first sentence reframes it:

> The real problem is not “find PII with regex”. It is that a bank cannot prove, at any moment, where each customer’s identifiers are, who can read them in clear, and which copy leaked. Detection, masking, tracking and erasure are four views of that one missing guarantee.

That reframing drives everything. Aran is not “a better regex”; it is a way to make protection *provable*.

The analysis then listed:

- **Five leak channels**: API payloads, application and debug logs (the worst, because every service writes them, they are kept long and many engineers read them), analytics exports, support tools and shared reports, and prompts sent to AI tools.
- **The structure of each data type** (A1): three carry a check digit (Aadhaar, card, and PAN’s private check letter); OTPs carry nothing.

- **A threat model** of four actors: careless code that logs whole objects, insiders who copy exports or paste into AI tools, vendors who receive shared data, and outside attackers. Because developers cannot be trusted to call a library correctly, the firewall must sit on the choke points data must cross.
- **Four hard constraints**: protection must be inline and fast; some systems need the real value back, so protection must be reversible for them and irreversible for everyone else; tracking must survive copies the bank does not control (a vendor database, a Telegram dump); erasure must reach every copy while respecting laws that force retention.

C3. Step 2: the baseline, and where it breaks

The baseline is what most teams would build: a detect-then-redact proxy. Intercept at the API gateway, log shipper, export path and AI gateway; detect with regex, checksums, context words and NER (Microsoft Presidio already has India recognisers); protect by masking, vault tokens or FF1 encryption; track with an audit log; erase by deleting rows the lineage lists.

It breaks in five places, and each became a research gap:

1. Number patterns fire on order IDs, timestamps and block IDs, so teams raise thresholds and miss real leaks; OTPs cannot be found by pattern at all.
2. A token or FF1 ciphertext looks exactly like a real number, so a scanner cannot tell a protected Aadhaar from a leaked one.
3. Masking is judged yes or no; nobody measures what a mask, or three masks together, still reveal.
4. Lineage stops at the bank's boundary: when a dump appears on Telegram, the audit log cannot say which of 40 vendors leaked it.
5. Deleting rows cannot reach immutable logs, backups, vendor copies or AI caches, so erasure is best effort, not proof.

C4. Step 3: the literature survey, and how each source was used

Rules (what the firewall must satisfy)

Source	What it says	How Aran used it
DPDP Act 2023, Schedule	Up to ₹250 crore for failed safeguards, ₹200 crore for failed breach notice	Masking and encryption are the named safeguards, so a leak without them is the costliest violation. Sets the stakes in the pitch and the "without Aran" demo.
DPDP Rules 2025 (notified 13 Nov 2025)	Duties from 13 May 2027; breach report in 72 hours; logs kept at least 1 year	Aran must give a 72-hour forensic answer (leak tracing), and erasure must coexist with log retention (purpose-bound keys).
CERT-In Directions, April 2022	Report incidents in 6 hours; keep ICT logs 180 days in India	Logs are kept by law, so they must be safe to keep: **protect before write**, not after.
UIDAI Aadhaar Data Vault, Circular 8 of 2025	Aadhaar only in a vault with HSM and AES-256; other systems use reference keys	Tokens are mandatory for Aadhaar anyway; Aran extends the reference-key idea to every sink.

Source	What it says	How Aran used it
PCI DSS and 8-digit BINs	First 6 and last 4 is the most a brand allows	The mask format is fixed by the card brands, so its residual risk must be *measured*, which led to E4 and M4.
NIST SP 800-38G Rev. 1 draft (Feb 2025)	FF3 removed; FF1 kept with a larger minimum domain	Use FF1 only, and never on tiny fields (OTP, last 4, CVV).

Real leaks (what fails in practice)

Incident	What happened	Lesson Aran took
Indian debit card breach, 2016	About 3.2 million cards from SBI, HDFC, ICICI, Yes and Axis via malware at a payment processor	Leaks happen outside the bank, so attribution needs evidence that survives the boundary (M3).
SBI server, 2019	An unprotected server exposed balances and transactions from SMS-service logs	Message and log stores leak, not only core databases.
Juspay, 2020	35 million records of masked card data, found when sold on the dark web	Masked data still sells, and breaches surface months later as outside dumps (M3, M4).
Star Health, 2024	31 million customers' records sold through Telegram bots	The first evidence is often a public dump, and the first question is which system it came from (M3).
Meta passwords in logs, fined 2024	Hundreds of millions of passwords logged in plain text; €91 million fine	Logs are a top leak channel even at the best-run firms.
Samsung and ChatGPT, 2023	Staff pasted source code and notes into ChatGPT	AI tools are a new exit path classic tools do not see (AI gateway).
IBM Cost of a Data Breach (India)	₹22 crore average in 2025; ₹25.5 crore in 2026, ₹40.9 crore for financial services	The cost argument for banks.

Research papers (what exists, and what it leaves open)

This is the "what papers did I use and how" table. Each row is a line of research, the key papers, what Aran took from it, and the gap it leaves that Aran fills.

Research line	Key papers	What Aran took	What it leaves open (Aran's opening)
Check-digit theory	Luhn 1960 (US patent 2,950,048); Verhoeff 1969; Gumm 1985 (IEEE Trans. IT)	The proof that Verhoeff catches every single-digit error and adjacent swap, and Luhn every single-digit error.	Check digits were only ever used for *input validation*, never as a security property. **This is the seed of M1.**
Format-preserving encryption	Bellare et al. 2009; NIST SP 800-38G (FF1); Durak and Vaudenay 2017 (FF3 broken); Beyne 2021 (FF3-1 attack)	FF1 as a secure way to make a 12-digit number from a 12-digit number; avoid FF3.	FPE output looks exactly like a real number, so scanners cannot tell safe from unsafe (gap G2).

Research line	Key papers	What Aran took	What it leaves open (Aran's opening)
PII detection in text	Microsoft Presidio; Pilán et al. 2022 (Text Anonymization Benchmark); Lison et al. 2021 (ACL survey)	The layered recogniser (regex + checksum + context + NER) as the baseline.	Scores are not calibrated probabilities; no OTP detection; numbers in logs raise floods of false alarms (G1, G6). **Seed of M2.**
PII leakage from LLMs	Carlini et al. 2021 (USENIX Security); Lukas et al. 2023 (IEEE S&P)	Models memorise and repeat PII; scrubbing reduces but does not remove it.	Assumes scrubbing is the defence; no reversible tokens for prompts. **Seed of the AI gateway.**
Taint and information-flow tracking	Enck et al. 2010 (TaintDroid); Arzt et al. 2014 (FlowDroid); Pasquier et al. 2017 (CamFlow)	The idea of following sensitive data through a system.	Needs every service instrumented, and stops at the network or vendor boundary (G4).
Leak attribution, traitor tracing, fingerprinting	Chor, Fiat and Naor 1994 (traitor tracing); Boneh and Shaw 1998, Tardos 2008 (fingerprinting codes); Juels and Rivest 2013 (honeywords)	Give each recipient a slightly different copy, so a leaked copy reveals who leaked it. Honeywords: plant values whose appearance proves a leak.	Built for films and content, not per-field identifiers in banking flows. **Seed of M3** (per-sink tokens) and of the "detect a secret because it was issued" OTP registry.
Source localisation on graphs	Shah and Zaman 2011 (rumour centrality); Pinto, Thiran and Vetterli 2012	Maximum-likelihood source of a spread from a few observations.	Never applied to a bank's data lineage. **Used in M3's fallback** for raw dumps.
Quantitative information flow	Smith 2009 (min-entropy leakage); Sweeney 2000 (87% unique by ZIP, sex, birth date)	Measure leakage in bits; small fields re-identify people in combination.	Not used to set masking policy, and no budget across many sinks (G3). **Seed of M4.**
Secure deletion and crypto-shredding	Boneh and Lipton 1996 (revocable backups); Perlman 2005 (Ephemerizer); Geambasu et al. 2009 (Vanish); Green and Miers 2015 (puncturable encryption)	Destroy a key and every ciphertext under it, including backups, is gone.	Breaks when one deterministic token is shared by many records; ignores legal retention (G5). **Seed of M5.**
Machine unlearning	Cao and Yang 2015; Bourtoule et al. 2021 (SISA)	Removing a person from a trained model is possible but expensive.	Better to keep identifiers out of models in the first place, which is what the AI gateway does.
Transparency logs	Laurie, Langley and Kasper 2013 (Certificate Transparency, RFC 6962); Crosby and Wallach 2009 (tamper-evident logging)	Append-only Merkle logs with inclusion proofs anyone can check.	Never used as erasure receipts. **Used in M5** and the AI gateway's audit chain.
Probabilistic filters	Bloom 1970; Fan et al. 2014 (cuckoo filter)	Compact set membership with a known error rate; the cuckoo filter can delete.	Used for the own-customer registry.
Artificial immune systems	Forrest, Perelson, Allen and Cherukuri 1994 (negative selection); Ji and Dasgupta 2004 (V-detector)	Learn "self", generate detectors that avoid it, flag "non-self".	Used for data-flow monitoring (the immune layer).

Research line	Key papers	What Aran took	What it leaves open (Aran's opening)
Post-quantum	NIST FIPS 203/204/205 (2024); NIST IR 8547; Mosca 2018; Gidney 2025; X-Wing hybrid KEM 2024; Kuwakado and Morii 2010 (Simon's attack on Feistel)	Which algorithms break, when to migrate, how to combine classical and post-quantum safely; why FF1 stays safe.	Nobody had pointed out that crypto-shredding can be undone retroactively (Aran's E17 finding).

Datasets used are listed in Appendix D. Every source is linked in the research reports.

C5. Step 4: measuring the gaps on real data

Instead of only arguing that existing approaches fail, the research measured it on the 32,000 real Loghub lines (E1) and with generated identifiers (E2 to E4). That produced six gaps, each with evidence:

Gap	Evidence	The research problem it creates
G1 Number detectors drown in false alarms	A plain OTP pattern fired **67,481** times on 32,000 lines; a card pattern **6,086** times; the word "otp" matched inside "hotplug"	Detection must be a calibrated decision with a provable false-alarm rate
G2 Protected and leaked values look the same	FF1 and vault tokens keep the 12 or 16 digit shape	Make protection itself detectable
G3 Masking is judged yes or no	First 8 and last 4 of a card leaves **1,000** candidates (10 bits); first 4 and last 4 of Aadhaar also 1,000	Measure what masks leak in bits, and cap what groups of sinks learn
G4 Leaks are attributed by access logs, not by the leaked data	Juspay and Star Health surfaced as outside dumps months later	The leaked bytes themselves must name the system that leaked them
G5 Erasure cannot reach logs, backups, vendors and AI caches, and clashes with retention	DPDP, CERT-In and RBI KYC rules all force retention	Erasure that reaches every copy in one operation, scoped by purpose, with proof
G6 OTPs have no structure	Any 4 to 8 digit number looks like an OTP	Detect a secret by what it *is* (an issued secret), not by how it looks

C6. Step 5: how the solution was derived

This is the chain of reasoning that produced Aran. Each link answers one gap.

Link 1: the key observation (G2 → M1). Aadhaar and card numbers already carry a check digit, and the check-digit literature proves that a real number always passes it. Everyone uses that only to catch typing mistakes. Turn it around: if every *token* is built to always *fail* the checksum, then "passes the checksum" means "real" and "fails" means "token", everywhere, with no ambiguity. The token keeps its shape (FF1 or a random body) and gets a deliberately wrong last digit, wrong by a keyed offset between 1 and 9.

- **Theorem 1 (perfect separability).** Every real Aadhaar passes Verhoeff; no Aran token does. *Why:* for any 11-digit body exactly one last digit passes, and the token's last digit differs from it by a non-zero amount.

- *Consequence:* any scanner, anywhere (a log platform, a data lake, a dump found online) can classify every 12-digit hit with zero ambiguity. Detokenising is still lossless, because the true check digit is recomputed from the decrypted body.

Link 2: once tokens are recognisable, make them name their sink (G4 → M3). Traitor tracing gives each recipient a different copy. Do the same per field: use the sink's name as FF1's tweak, so the same customer has a different token in every destination. A leaked token then names the system it was issued to.

- **Theorem 2 (attribution).** The chance that a leaked token is attributed to the wrong sink is at most (S − 1)·N / D per token, and that to the power k for a dump of k tokens. For a bank with 50 million customers and 40 sinks, one token is wrong at most 2% of the time, a dump of 5 tokens at most 3 in a billion; in vault mode it is zero.
- *Fallback (designed, not built yet):* if a dump holds raw values (leaked before Aran, or by someone allowed to detokenise), rumour-source localisation picks the most likely source: the smallest system whose holdings contain the whole dump. Every detokenise call is logged per caller (that part is built), so an insider's raw dump can be matched to their log.

Link 3: masks leak bits, and joins leak more (G3 → M4). Quantitative information flow measures leakage as min-entropy: log2 of the number of identifiers still consistent with what a sink has seen. Aran counts this exactly with a dynamic programme over the checksum. The new step is to keep a *budget per trust domain* (all systems that can join their data), not per field, and refuse any disclosure that would push a customer below a threshold such as 20 bits.

Link 4: numbers without structure (G1, G6 → M2). Treat detection as a decision, not a pattern match. Each clue (checksum, issuer prefix, context words, NER score, registry hit) is a likelihood ratio, multiplied together by Bayes' rule; by the Neyman-Pearson lemma, thresholding the result is the best possible test at the false-alarm rate the bank chooses. The strongest clues come from what the bank already owns:

- an **own-customer registry**: keyed hashes of every identifier the bank holds, in a Bloom (later cuckoo) filter, so a hit is near-certain evidence;
- an **issued-OTP registry bound to the request's trace ID**: inspired by honeywords, an OTP is detected because it was issued in *this* request. Without the binding, a random 6-digit number matches one of 70,000 live OTPs 7% of the time; with it, one in a million.

Link 5: erasure that reaches every copy, and still obeys retention (G5 → M5). Crypto-shredding destroys a key to destroy every copy, but fails when one deterministic token is shared, and ignores legal holds. So: one key per customer **per purpose** (servicing, marketing, fraud, KYC legal hold); every sink sees only **random** vault tokens; KYC keys are time-locked and destroyed automatically when retention ends; every key destruction becomes a leaf in a Merkle transparency log, and the customer gets a signed inclusion proof as a receipt.

- **Theorem 3 (erasure by key destruction).** Once the key and the vault index for a customer are destroyed, every copy of their token anywhere is statistically independent of their identifier. *Why:* the token was random; its only link to the identifier was the vault entry, now ciphertext under a destroyed key (Shannon perfect secrecy).
- *Later refinement (post-quantum research, E17):* this holds **only if no copy of the key ever travelled under quantum-breakable key exchange**. Otherwise an attacker who recorded that traffic can recover the key later. Hence hybrid post-quantum key transport.

Link 6: AI where maths cannot reach. Names and addresses have no checksum, so NER is needed; OTPs and UPI IDs need context; and all clues need calibrating so the probabilities mean something. AI is used exactly there, and nowhere a checksum gives an exact answer.

Link 7: watching flows, not just values (the immune layer). Even a perfect detector cannot see an insider who detokenises thousands of records at 2 a.m., or a new unknown destination. The body's immune system solves "attack anything unfamiliar, but never yourself" with negative selection. Applied to data flows (which system sends which data class to which sink, how much, when, how much raw), it flags unusual flows without needing examples of attacks.

Link 8: the AI tools channel. The same tokens protect prompts: identifiers become typed placeholders such as `<AADHAAR_1>`, the model works on placeholders, and the reply is filled back in only for staff whose role allows it. The model provider never receives a raw identifier, and anything it memorises is traceable to the AI gateway.

C7. Step 6: the critical review

The design was then attacked. Eleven weaknesses were found, and every fix became part of the design (most are now built):

Attack or weakness	Severity	Fix
An insider with detokenise rights leaks raw values, so no token names the sink	High	Log every detokenise call per caller (built; the immune layer also watches detokenise rates); match a raw dump to the caller whose log contains it (designed)
A real Aadhaar with one typo fails Verhoeff and looks like a token	High	Typo repair: try the 108 one-digit edits and 11 swaps against the own-customer registry (built, E12: 99.2%)
Vaultless FF1 tokens cannot be crypto-shredded, because one sink key decrypts every customer	High	DPDP-scope data uses random vault tokens only (built in `shredder.py`)
A Bloom filter cannot delete, so erased customers stay in the registry	Medium	Cuckoo filter (built)
The registry is a list of every customer	Medium	Entries are HMACs under an HSM key; lookups are rate-limited and logged
Identifiers hidden by encoding	Medium	The normaliser (built, E10, E15)
Bayes multiplies correlated clues and becomes overconfident	Medium	Calibration with isotonic regression, ECE reported (built)
Min-entropy assumes identifiers are uniform; real ones are not	Medium	Treat ledger bits as an upper bound and keep a margin
OTP binding needs a trace ID in the log line	Medium	Fallback to context rules and the classifier (built)
FF1 on tiny fields (OTP, last 4, CVV) is weak	Low	Never FPE a domain under 10^6; tiny fields are vaulted or destroyed (OTPs are destroyed)
Crypto-shredding is not named in DPDP as erasure	Legal	The argument is Theorem 3; needs a lawyer's view (open)

C8. What is genuinely new, stated honestly

Some payment tokenisation products already offer card tokens that fail Luhn, so "a token that fails the checksum" alone is not new. What the research claims as new is the combination, each part with a stated theorem rather than a heuristic:

1. Extending checksum-failing tokens to Aadhaar's Verhoeff code with a keyed per-sink offset, and using the checksum as a **proven detection invariant across the whole data estate** (Theorem 1).
2. **Per-sink tokens whose leaked copies name their source**, with an explicit error bound (Theorem 2), plus a maximum-likelihood fallback for raw dumps.
3. A **trace-bound issued-secret registry** that makes structureless OTPs detectable.
4. A **min-entropy leakage ledger** that budgets what *groups* of sinks learn, not single fields.
5. **Purpose-bound, time-locked crypto-shredding with verifiable Merkle receipts**, reconciling DPDP erasure with legal retention (Theorem 3).
6. Added later: the finding that **crypto-shredding can be undone retroactively** unless keys only ever travelled under quantum-safe key exchange, and the hybrid fix (E17).

No surveyed paper or product joins these. A workshop paper and a legal opinion are on the roadmap (item 13).

Part D. How Aran works, end to end

D1. Where Aran sits in a bank

Aran is a **backend product**. Customers never see it; the bank's IT team installs it like a firewall. It sits on the choke points every piece of data must cross:

```
SOURCES               ARAN ENGINE (inline, before anything is stored)      SINKS
                      +-------------------------------------------------+
Mobile app   --API--> | 1 Normalise   undo disguises                    |
Core banking --API--> | 2 Detect (M2) checksum + AI + registries        |   log platform
Loan system  --API--> | 3 Decide (M4) leakage ledger                    | --> CRM, analytics
Log files    --log--> | 4 Protect     M1 tokens, per sink (M3)          |   vendors
Exports    --export-> | 5 Record      flow -> immune layer -> alerts    |   AI provider
Staff + LLM  --AI---> |                                                 |   (tokens only)
                      +-------------------------+-----------------------+
                                                |
                  +-----------------------------+-------------------------------+
                  | Vault + keys (M5): one key per customer per purpose, in an HSM   |
                  | Merkle log: every erasure, with a signed receipt to the customer |
                  +------------------------------------------------------------------+

If a copy leaks --> Leak trace: a token names its sink (M3);
                    a raw dump is matched to the most likely holder or detokenise caller
```

In the prototype, the "API gateway" and "log shipper" are the FastAPI server's `/api/process` endpoint and the firewall's `process(line, source, sink)` call; the AI gateway is a real OpenAI-compatible endpoint (`/v1/chat/completions`).

D2. One request, followed through every layer

A customer, Ravi (a made-up person), applies for a loan in the mobile app. The app sends:

```
name: Ravi Kumar, Aadhaar: 2345 6789 0124, PAN: ABCPK1234L, mobile: 98765 43210,
address: 12 Gandhi Street, Madurai 625001, OTP 482913 (trace 4bf92f...)
```

This message is going to the **loan vendor** (an outside company) and its log line is going to the **log platform**. Here is what Aran does, step by step.

1. **Normalise** (`research/firewall/normalise.py`). The text is turned into "views" that undo disguises: Tamil or Devanagari digits become 0 to 9, `%34` becomes `4`, `2345.6789.0124` and `2 3 4 5 ...` lose their separators, base64 is decoded, split fields `aadhaar_1=2345 aadhaar_2=6789 ...` are joined. Every character remembers where it came from, so a finding can be replaced in the original text.
2. **Detect** (`research/ai/detector.py`, `context_clf.py`, `ner.py`). Candidates are proposed by type. `234567890124` passes Verhoeff (strong evidence); `ABCPK1234L` has PAN's exact shape; `9876543210` starts with 9 and follows the word "mobile"; `482913` follows "OTP" and, if the OTP registry is on, its keyed hash with this request's trace ID is live; "Ravi Kumar" and the address are found by the CRF NER. Each clue is a likelihood ratio; a per-type model fuses them, and calibration turns the score into a real probability. Anything above the sink's threshold is sensitive.
3. **Decide** (`research/firewall/ledger.py`). If the sink asked for a partial mask (say "last 4 digits of the card"), the leakage ledger checks what this vendor's other systems have already seen of this customer, and refuses or shrinks the mask if the combination would leave fewer than about a

million candidates (20 bits).

4. **Protect** (`research/firewall/aran_firewall.py`, `shredder.py`). Each type is handled differently:
   - Aadhaar and card become **checksum-complement tokens issued to this sink** (12 or 16 digits, guaranteed to fail the checksum). For data in DPDP scope, the token is random and stored in the purpose vault, encrypted under Ravi's key for the "loans" purpose.
   - PAN, mobile, UPI, name and address become **keyed per-sink pseudonyms** like `<PER:219779fa46>`.
   - The **OTP is destroyed**, never stored anywhere, because no one downstream ever needs it.
5. **Record** (`app/server/engine.py`). The event (source, sink, types found, how many left raw, latency) is stored, and the **immune layer** adds it to the current traffic window for that source and sink.

The vendor receives something like:

```
name: <PER:219779fa46>, Aadhaar: 0980 6158 6412, PAN: <PAN:77816c3651>, mobile: <MOBILE:5c1e...>,
address: <ADDR:a07b...>, OTP [removed]
```

The loan system still works: every field has the right shape. Only a system with permission can call `/api/detokenise` to get Ravi's real Aadhaar back, and every such call is logged with the caller's name.

Later, three things can happen:

- **The vendor leaks.** A file with `098261586412` appears on Telegram. Anyone can check that it fails Verhoeff, so it is a token, not a raw Aadhaar; Aran's trace says it was issued only to the loan vendor. In the console, the Leak trace page does exactly this with what a sink really stored.
- **Ravi asks for erasure.** The bank destroys Ravi's "loans" and "marketing" keys. Every copy of his tokens, in every log, backup and vendor database, is now meaningless. Ravi gets a receipt signed with Ed25519 and ML-DSA-65, with a Merkle inclusion proof that anyone (including his phone's browser) can check. If he asks to erase the "KYC" purpose before the legal retention ends, Aran refuses with a clear reason (HTTP 423, "locked"), and destroys it automatically when the period ends.
- **An insider misbehaves.** At 2 a.m., a staff account detokenises 3,000 records. No single rule fires, but the combination of hour and detokenise rate is far from anything in "self", and the immune layer raises an alert.

D3. The five core mechanisms in depth

M1. Checksum-complement tokens

- **Code:** `cc_token_aadhaar` in `research/aran_core.py`; `cc_token_card` in `firewall/aran_firewall.py`; vault-mode `random_cc_token`.
- **How:** take the 11-digit body, encrypt it with FF1 using the sink name as tweak (or draw a random body in vault mode), compute the correct Verhoeff digit for the new body, then add a keyed offset between 1 and 9 so the final digit is always wrong. Cards do the same with 15 digits and Luhn.
- **Why it matters:** "passes = real, fails = token" holds everywhere, so any scanner can tell protected data from leaked data. Measured: 20,000 tokens injected into real logs, 0 flagged; 20,000 raw values, all flagged (E2).

M2. Evidence-fusion detector with registries

- **Code:** `research/ai/detector.py` (fusion and calibration), `ai/context_clf.py` (OTP/UPI), `ai/ner.py` (names and addresses), `firewall/registry.py` (registries, typo repair).
- **Evidence per candidate:** checksum result, issuer prefix, context classifier score, NER probability, nearby keywords, date words, PIN code, phone-named JSON fields, words that introduce reference numbers, and registry hits.
- **Hard gate:** an Aadhaar or card candidate that fails its checksum gets probability 0 by design (it is a token or not an identifier), unless typo repair finds a registered customer one edit away. A second check makes sure Aran never mistakes its *own* token for a typo (without it, 1.5% of tokens could look like a typo of some customer at a bank with 100 million registered numbers).
- **Registries:** HMAC fingerprints in a cuckoo filter (32-bit fingerprints, false-positive rate about 2 in a billion; 800,000 entries take 4 MB). OTPs are registered as HMAC(key, OTP | trace ID) with a time-to-live.

M3. Per-sink tokens and leak attribution

- **Code:** the sink name is the FF1 tweak and selects the offset; `/api/trace` in the server; E3 in `research/experiment.py`.
- **How:** given a dump, Aran looks each token up in the vault or in the index of tokens it issued, or, for an FF1 token, finds the one sink whose keyed offset explains its wrong check digit and whose decryption is a registered customer. Each match is a vote for a sink, and the sink with the most votes is named. Any raw Aadhaar or card numbers in the dump are counted separately, because they escaped protection (the research design's maximum-likelihood fallback for raw dumps is not built yet).
- **Measured:** 0 collisions among 800,000 tokens; the right sink for 100% of dumps of 1, 2, 5 and 50 tokens among 40 sinks (E3).

M4. Leakage ledger

- **Code:** `research/firewall/ledger.py`; console page `#ledger`.
- **How:** for every identifier and every trust domain (systems that can share data, such as all of one vendor's systems), it remembers which digit positions have been shown, counts exactly how many valid identifiers still fit, and refuses any disclosure that would go below the threshold. A refused mask falls back to a smaller mask or a token. Entries are keyed by HMAC, so the ledger itself holds no identifiers.
- **Measured:** with masks as configured, 14,175 identifiers ended up below the threshold in some vendor's joined view; a per-field check fixed almost none of them (13,053 left); the ledger brought it to **0**, by shrinking 9.7% of disclosures (E13).

M5. Purpose-bound crypto-shredding with receipts

- **Code:** `research/firewall/shredder.py` (`PurposeVault`, `MerkleLog`, `verify_receipt`, `PurposeFirewall`); tests in `research/test_shredder.py`.
- **How:** an AES-256-GCM key per customer per purpose; vault tokens are random and fail their checksum; erasure destroys the key and the index entries; each destruction is appended to an RFC 6962 Merkle log; the receipt is signed with Ed25519 and ML-DSA-65 over the same bytes, with the algorithm name inside the signed bytes so the post-quantum half cannot be stripped. KYC keys are locked until their retention ends.

- **Measured:** after erasure the bank itself reads 0 of the erased values; a tampered receipt fails verification; the browser re-checks the inclusion proof independently.

D4. The added layers in depth

AI layer (owner pair: Bharadhwaj, Arun).

- *NER* (`ai/ner.py`): a CRF with features that understand both Latin and Devanagari script. Trained on CoNLL-2003, HiNER, Hinglish tweets and generated bank log lines. A cheap **gate** (`may_have_entities`) sends only lines that might contain a name to the NER, which doubled speed with no loss.
- *Context classifier* (`ai/context_clf.py`): character n-grams plus a small lexicon and the published UPI handle list; tells an OTP from a port or order number, a UPI ID from an e-mail or `user@host`.
- *Calibrated detector* (`ai/detector.py`): one stacker per type, isotonic calibration, trained so the calibration reflects unseen phrasings ("fold" model trained on two thirds, calibrated on the remaining third).

Immune layer (owner pair: Inbathamizhan, Aasif). `research/immune/ais.py` (negative selection with V-detectors, memory, tolerance, self-radius tuning) and `immune/flows.py` (simulated telemetry for 9 bank data flows). Each traffic window (source, sink, hour) becomes a point with 7 features, relative to that pair's own normal, so 2 a.m. is normal for the nightly warehouse load but not for the loan vendor. In the live console it learns from replayed real Loghub logs and the "drills" inject attacks you can watch it catch.

Normaliser and red-team additions (cyber pair). Besides the 12 E10 tricks, it can (off by default) read numbers written as words in English, Hindi, Tamil and Hinglish (with "double" and "triple"), look-alike letters (O for 0, I for 1, S for 5, B for 8), and double escapes.

AI gateway (`research/gateway/ai_gateway.py`; both pairs). OpenAI-compatible. Values become typed placeholders (`<AADHAAR_1>`) before the provider sees them; replies are scanned again; placeholders are filled back only for the conversation's own caller and only for types that caller's role may see (customer care sees mobile and UPI but only the last 4 of Aadhaar and card; OTPs are never stored); every resolution goes into a hash-chained audit log. Set `ARAN_LLM_URL`, `ARAN_LLM_KEY` and `ARAN_LLM_MODEL` to use a real model; without them it uses a local echo stand-in and no data leaves the machine.

Post-quantum (`research/pqc/hybrid.py`, `pqc/cbom.py`; cyber pair). Hybrid signatures and key sealing, a CBOM scanner and a Mosca priority table.

D5. Code map

Folder or file	What it is	Owner pair
`research/aran_core.py`	Verhoeff, Luhn, NIST FF1, checksum-complement tokens, min-entropy counting	Cyber
`research/synth.py`	Generated test identifiers and bank log lines (valid checksums, belong to no one)	All
`research/ai/`	NER, OTP/UPI classifier, calibrated detector	AI/ML

Folder or file	What it is	Owner pair
`research/immune/`	Immune layer and flow simulation	Cyber
`research/firewall/`	The wired firewall, normaliser, registries, ledger, shredder	Cyber
`research/gateway/`	AI gateway	Both
`research/pqc/`	Post-quantum hybrids and CBOM	Cyber
`research/experiment*.py`, `results*.json`	E1 to E19 and their raw numbers	All
`research/test_*.py`	Tests (including the NIST FF1 sample vectors)	All
`app/server/`	FastAPI server: `engine.py` (the live engine), `main.py` (routes), `standout.py` (demo features), `build_models.py`	Cyber + AI/ML
`app/ui/`	Main console: your design, wired to the server, no build step	All
`app/web/`	Earlier React console at `/classic`, with the 3D scenes	All
`docs/research/`	The six research reports	
`docs/learning/`	Learn 1 to 20	
`artifacts/`	PDFs, screenshots, design spec, prompts, this document	
`notes/`, `Home.md`, `templates/`, `.obsidian/`	The Obsidian vault	
`ui-design/`	Your original static design (before wiring)	
`site-research/`	Notes on the five reference websites	
`iravora-void/`	Separate practice site, not part of Aran	
`Dockerfile`	Builds and serves everything from one container	

Part E. Every experiment and what it proved

Every experiment has a script in `research/` and writes its numbers to a JSON results file, so anyone can re-run it. No real person's identifier is used anywhere.

E1. The experiments, one table

#	What it tested	Data	Result	What it means
E1	False alarms of plain patterns vs Aran's checks	32,000 real Loghub lines, 16 systems (no Indian IDs, so every hit is false)	OTP pattern 67,481 → context rule 6; card pattern 6,086 → Luhn + issuer prefix 81; Aadhaar pattern 366 → Verhoeff 4	Checksums and context cut false alarms by 90 to 10,000 times.
E2	Can tokens be told apart from real numbers?	20,000 tokens and 20,000 raw values injected into real logs	0 tokens flagged; 20,000 raw values all flagged	Theorem 1 holds in practice.
E3	Does a leaked dump name the right sink?	20,000 customers, 40 sinks, 200 trials per dump size	0 collisions among 800,000 tokens; 100% right for dumps of 1, 2, 5 and 50 tokens	Theorem 2 holds in practice.
E4	How much do common masks leak?	Exact counting	Card last 4: 36.5 bits; Aadhaar last 4: 22.9; card first 6 + last 4: 16.6; card first 8 + last 4: **10.0** (1,000 candidates); Aadhaar first 4 + last 4: **10.0**	Masks that look safe can leave only 1,000 candidates, guessable in seconds.
E5	Names and addresses (NER)	CoNLL-2003, HiNER, Hinglish tweets, generated bank log lines	Person F1: 0.85 English news, 0.76 Hindi, 0.69 Hinglish tweets; on bank log lines 0.85 / 0.77 / 0.83; address F1 0.88 to 1.00	A laptop-sized model handles three languages; log text needs log-text training.
E6	OTP and UPI context classifier	Unseen phrasings, 8 unseen real systems, real SMS	OTP precision 1.00, recall 0.80 (regex 0.61); UPI ranking AUROC 1.00	Context beats patterns; raw thresholds need calibration on new systems.
E7	Calibrated detector	45,000 test candidates	Calibration error 0.015 (later 0.0065); false alarms on 16,000 real lines 2,506 → **60**	Probabilities mean what they say, and false alarms collapse.
E8	Immune layer	Simulated telemetry of 9 bank flows over 30 days, 6 attack scenarios	Immune 100% caught at 0.6% false alarms; static rules 90%; Isolation Forest 20%; memory lifted a repeat attack's score 0.37 → 0.71	Catches the insider static rules miss and the subtle leaks Isolation Forest misses.

#	What it tested	Data	Result	What it means
E9	The whole firewall on real logs	16,000 real lines from 8 unseen systems, 1,895 injected values	Values removed 83.5% (regex 46.1%); clean lines wrongly changed 0.8% (regex flagged 15.8%); tokens re-detected as real: 0	The wired system works on data it never saw.
E10	Identifiers hidden 12 ways	700 test lines, 783 values	Caught 8.3% → 80.3% (normaliser) → **100%** (with registry); no extra false alarms	Disguises do not work.
E11	Registries in the full run	Same as E9, with 800,000-entry registry and 70,000 live decoy OTPs	Values removed 89.7%; mobiles and OTPs 100% (mobiles 94.2% if only 70% are own customers)	The two weak spots close for a bank that registers its customers and logs trace IDs.
E12	Typo repair	1,990 registered identifiers with one typo or swap	0% → **99.2%**; 0 tokens wrongly flagged as typos	A mistyped real number is still caught.
E13	Leakage ledger	20,000 customers, 11 sinks in 4 trust domains, 167,654 disclosures	Identifiers below 20 bits in a joined view: 14,175 → **0** (per-field checks: 13,053)	The leak is in the *join*, and only a cross-system budget stops it.
E13b	Do the mobile/OTP fixes generalise?	12 phrasings written after the fix	Mobile recall 67.2% → 100% but 17.3% of "cheque no" numbers wrongly taken as mobiles; OTP recall 94.3% → 84.7%	Two honest regressions to fix next.
E14	AI gateway	2,000 staff prompts to a recording provider; 5 attack types	Raw values reaching the provider 100% → **7.8%** (all names/addresses; every number type 0%); 0 of 5 attacks leaked; editing one audit entry detected	Staff can use chatbots without sending identifiers out.
E15	Red team	500 lines × 26 hiding tricks × 625 values	Fully caught 14 → **22 of 26**; two bugs found and fixed	Attacks measured, not assumed.
E16	Speed	E9 run	274 → 348 (cached features) → **654** lines/second (NER gate)	Still far from production speed.
E17	Retroactive un-erasure	1,000 customers, 1,200 erased values, simulated quantum break of X25519	Classical backup: **100%** recovered; hybrid ML-KEM: **0%**	Crypto-shredding needs quantum-safe key transport.

#	What it tested	Data	Result	What it means
E18	Cost of post-quantum	Benchmarks	Receipt 904 → 8,155 bytes; verifications 5,657 → 1,930 per second; FF1 about 3% slower with AES-256	Bigger, not much slower.
E19	CBOM and Mosca table	Aran's own code	Quantum-broken algorithms without a partner: 3 uses → 0; Aadhaar and PAN exposed for decades in every scenario	Migrate identifiers with the longest shelf life first.

E2. How the headline number moved

The main end-to-end number is "sensitive values removed by the firewall" on E9's 16,000 unseen real lines:

Stage	Values removed	Clean lines wrongly changed
Plain regex prototype	46.1%	2,357 flagged
AI layers (E9, first run)	83.5%	122
+ normaliser and registries (E11)	89.7%	122
+ mobile and OTP fixes, no registries (E13 thread)	91.0%	150 (130 with the NER gate)

Aadhaar, card, PAN and UPI have been at 100% since the first AI run. What is left is mostly names (about 77%) and addresses (about 90%).

E3. Weak spots and negative results, stated plainly

These are part of the result, and judges respect them. Know them before anyone asks.

1. **Names are the weakest type** (about 77% caught end to end), and every value that reached the AI provider in E14 was a name or address. Fix: transformer NER (roadmap item 7).
2. **Two regressions after the mobile/OTP fix** (E13b): "{OTP} is the PIN to authorise txn {id}" is missed; "cheque no {number}" is taken as a mobile. Fix: adjust features and test on a third, fresh set of phrasings.
3. **The immune layer failed on raw log lines** (BGL supercomputer logs: AUROC 0.34, worse than chance), because most test lines used log templates never seen in training, so "new" did not mean "attack". It is used only on data *flows*, where it works. Fix: parse logs into templates first (roadmap item 11).
4. **Immune tolerance generalised poorly**: marking false alarms as self did not reduce later false alarms.
5. **The flow attacks are simulated**, because no bank publishes its flow telemetry.
6. **The generated test lines were written by the same team as the training lines**, so scores on them are an upper bound until a bank's own (consented, masked) logs are used.

7. **Speed** is 654 lines per second in Python; a bank needs thousands per core. Fix: Rust or vectorised port (item 12).
8. **Four red-team tricks are left open on purpose**: letters between digits, reversed digits, hexadecimal and nested base64. Undoing them on every line would flag ordinary numbers; the immune layer watching the flow is the better defence there.
9. **The AI gateway was tested against a recorder, not a real model.** A real model might rephrase placeholders.
10. **No quantum computer was used**; E17 simulates Shor by handing the attacker the private key. It shows the attack path, not quantum hardware.
11. **Crypto-shredding as DPDP erasure needs a lawyer's view.**
12. **The live server forgets customers and keys when it restarts**; it is a demo, not a production deployment.

Part F. The live console and the demo

F1. What it is

`app/` is a working website that runs Aran's real code. Nothing shown on it is hard-coded: every number, token, receipt and alert comes from the server at that moment. Only the customers' identifiers are generated, because real ones must never be used. There are two frontends on the same server:

- `/` — the main console, your own design from `ui-design`, wired to the live API (plain HTML, CSS and JavaScript; GSAP, Lenis and three.js are bundled in `app/ui/vendor/`, so it works offline except for fonts).
- `/classic` — the earlier React console, with the 3D firewall and 3D immune scenes.

F2. The guided tour (13 chapters)

#	Chapter	What you see (all live)
1	A firewall for data	Real requests fly from bank systems into the Aran shield and come out as tokens (3D, with a 2D map on phones); you can type your own message
2	Leaks come through side doors	Real Indian breaches and their cost (fixed text from the research)
3	Why simple search fails	300 real log lines: plain filters light up red, Aran stays almost empty
4	The last digit is a lock	Verhoeff and Luhn running digit by digit; change one digit and watch it fail
5	Building a token	The 10 real FF1 rounds, then the deliberately wrong last digit; the server confirms the live firewall issued exactly that token
6	Finding what is sensitive	A scan beam, the clues as bars, a probability gauge
7	Disguises do not work	One Aadhaar hidden 12 ways; every card flips to "caught"
8	A leaked file names its source	A leaked file flies to the dark web; a trace beam names the sink
9	Erase everywhere, with proof	The purpose key shatters; KYC is refused; your browser re-checks the Merkle receipt itself; a tampered copy fails
10	An immune system for data flows	A radar of live traffic against the learned normal range; drills appear as alerts; the 3D feature space
11	Safe AI chatbots	What the AI provider sees (placeholders) and what each staff role gets back

#	Chapter	What you see (all live)
12	The evidence	The results, read live from the result files
13	Glossary and team	

F3. The console pages

Overview (live stats and events), **Live traffic** (send sample or your own requests, see before and after and why each value was caught), **Customers** (create customers, detokenise, erase, verify receipts, the Merkle log), **Leak trace** (load what a sink really stored and trace it), **Immune layer** (learn normal traffic from replayed real logs, run attack drills, confirm or dismiss alerts), **Results** (run the real test suite and fast experiments on the server), **How it works**.

Demo features: Beat the firewall (phones + QR + speech), Without/With Aran, Leakage ledger, Card photo scan, Post-quantum, My data, Tamil and Hindi, Presenter mode (described in B15).

F4. How to run it

```
cd research && ./fetch_data.sh && cd ..            # downloads the real Loghub logs
pip install -r app/server/requirements.txt
cd app/server && uvicorn main:app --port 8000       # open http://localhost:8000
```

- First load: a “Starting Aran” screen shows while the models load.
- For the immune demo, open **Immune layer** and click “Learn normal traffic” first; it needs some minutes of replayed traffic before drills give meaningful alerts.
- For the phone arena, start the server with `--host 0.0.0.0` and put the phones on the same Wi-Fi or hotspot.
- If the page looks old, press **Ctrl+F5**. `localhost` only works on the laptop running the server.
- On your laptop, the demo copies are `D:\Aran-live` and `D:\Aran-demo`; `D:\Aran` is the working copy and vault.
- Presenter mode: open `http://localhost:8000/?present=1` (Learn 20 has the demo-day checklist).

Part G. What is not done, and the future

G1. Roadmap status

#	Item	Status
1	Live gateway and dashboard	**Done** (live console)
2	Own-customer and OTP registries	**Done** (E11)
3	Fix mobile and OTP weak spots	**Done** (E9, E13b), with two regressions to fix
4	Crypto-shredding with erasure receipts	**Done** (`shredder.py`)
5	Normaliser and typo repair	**Done** (E10, E12)
6	AI gateway for LLM prompts	**Done** (E14); needs a real-model test
7	Transformer NER in a cascade	**Gate done** (E16); transformer not yet
8	Tamil (and other Indic) NER	Not started (the website speaks Tamil; the *detector* does not yet read Tamil names)
9	Red-team evaluation	**Done** (E15)
10	OCR on KYC images	**Demo page done** (`#scan`, Tesseract); not measured as research yet
11	Immune layer on log templates	Not started
12	Speed (Rust or vectorised)	First step (274 → 654 lines/s); port not started
13	Paper and legal review	Not started
14	Post-quantum upgrade	**Done** (E17 to E19); hybrid TLS and SLH-DSA still to do
M4	Leakage ledger	**Done** (E13, `#ledger`)

G2. Open to-do list

From the Obsidian notes and the latest threads:

- ☐ Fix the two E13b regressions and measure on a third set of fresh phrasings.
- ☐ Run the AI gateway against a real model (`ARAN_LLM_URL`, `ARAN_LLM_KEY`, `ARAN_LLM_MODEL`).
- ☐ Transformer NER for names behind the existing gate (item 7; read Learn 9).
- ☐ Tamil NER with the Naamapadam dataset, and Tamil OTP and address phrasings (item 8).
- ☐ Measure OCR on KYC images, with Verhoeff to catch misreads (item 10; read Learn 10).
- ☐ Immune layer on log templates with the Drain parser, and tolerance by affinity (item 11).
- ☐ Rust or vectorised NumPy port of the checksum, FF1 and regex stages (item 12; target 10,000 lines/s per core).
- ☐ Build the raw-dump maximum-likelihood source estimator and the detokenise-log join (designed in M3).

- ☐ Hybrid TLS (X25519MLKEM768) in front of the console, proxy and AI gateway; sign Merkle roots with SLH-DSA.
- ☐ Persist customers, keys and the Merkle log so the server survives a restart (the demo keeps them in memory).
- ☐ Public hosting: give Render access to the Aran repo (Render → Account settings → GitHub → Configure), then deploy from the `Dockerfile`.
- ☐ Workshop paper on Theorems 1 to 3 and E1 to E19; a short legal opinion on crypto-shredding as DPDP erasure (item 13).
- ☐ Housekeeping: set the GitHub repo description ("Detect and automatically mask sensitive customer data in banking API traffic and application logs before it is stored or shared."), and delete any leftover old work branches on GitHub.

G3. Suggested plan per pair

	Cybersecurity pair (Inbathamizhan, Aasif)	AI/ML pair (Bharadhwaj, Arun)
Next 1 to 2 weeks	Persistence for the vault and Merkle log; raw-dump source estimator; hybrid TLS in front of the server	Fix the E13b regressions; transformer NER behind the gate; real-model AI gateway test
After that	Rust port of the hot path (item 12); immune layer on templates (item 11)	Tamil NER (item 8); OCR measurement (item 10)
Together	Rehearse the demo in presenter mode; draft the paper and get the legal view (item 13)	

G4. The product vision

What a bank would buy: a gateway server (the FastAPI app, packaged with Docker) placed on its internal traffic; a log cleaner that scrubs logs before they are stored or sent to vendors; an AI gateway in front of staff chatbots; a dashboard for security and compliance staff (what was caught, leak traces, alerts, erasure requests); and audit proof (signed erasure receipts, CBOMs, reports for RBI or the Data Protection Board).

How it could be sold: on-premise software licensed per bank (banks prefer data never leaving their data centre), or a managed service for smaller banks, NBFCs and fintechs.

Why now: DPDP Rules duties start 13 May 2027 with penalties up to ₹250 crore; CERT-In and DPDP force logs to be kept, so they must be safe to keep; RBI's FREE-AI framework asks banks to govern data going into AI; India's quantum-safe roadmap asks banks to migrate by 2033 and vendors for CBOMs from 2027.

What it would take to go from demo to pilot: the Rust hot path, persistence and an HSM or cloud KMS for keys, adapters for real choke points (an Envoy or Kong filter for APIs, a Vector or Fluent Bit transform for logs), a partner bank's consented masked logs to retrain and re-measure, and the legal opinion.

Part H. Explaining Aran to others

H1. The 30-second version

> Banks leak customer data through logs, vendors and AI chatbots, not just databases. Aran is a firewall for data that sits on those paths. It finds Aadhaar, PAN, cards, UPI IDs, OTPs, names and addresses, and swaps them for look-alike tokens before anything is stored or shared. Our trick: every Aadhaar and card number has a check digit, and our tokens are built to always fail it, so anyone can prove whether a number is protected or leaked. Each destination gets different tokens, so a leaked file names its source. And each customer's data is locked under its own key, so erasing a customer is one key deletion with a signed receipt.

H2. The 3-minute demo path

Use presenter mode (`?present=1`, Learn 20): (1) tour chapter 1, packets turning into tokens; (2) chapter 3, plain filters drowning in false alarms on real logs; (3) chapter 5, a token built live and failing its checksum; (4) chapter 8, a leaked file naming its sink; (5) chapter 9, erasure with a receipt checked in the browser; (6) Beat the firewall, with judges on their phones; (7) close on the evidence chapter.

H3. Simple analogies that work

- **The security guard on every pipe**: Aran reads everything passing and swaps secrets for look-alikes.
- **A lock on the last digit**: real numbers always open the lock; Aran's tokens never do.
- **Marked banknotes**: each vendor gets differently marked copies, so a leaked copy shows who leaked it.
- **Shredding the only key**: delete one key and every copy everywhere becomes unreadable.
- **The body's immune system**: it learns what "self" looks like and reacts to anything foreign.

H4. Questions judges will ask, and the answers

Question	Answer
Is this just regex?	No. Regex raised 67,481 OTP false alarms on 32,000 real lines; Aran's checks raised 6. Numbers are confirmed with check-digit maths, words with calibrated AI models, OTPs by being issued.
Where is the AI?	Names and addresses (CRF NER in English, Hindi, Hinglish), OTP and UPI context (classifier), calibration of every finding, and protecting prompts to AI chatbots. Where a checksum gives an exact answer, we use maths on purpose.

Question	Answer
What is novel?	Section C8: tokens that provably fail the checksum (Theorem 1), per-sink tokens that name the leaker (Theorem 2), the trace-bound OTP registry, the cross-system leakage ledger, purpose-bound crypto-shredding with receipts (Theorem 3), and the retroactive un-erasure finding. Luhn-failing card tokens alone exist already; the combination and the proofs do not.
Did you use real data?	Real public logs (Loghub, 32,000 lines from 16 systems), real NER corpora and real SMS. Never real identifiers: those are generated with valid checksums, because using real ones would itself be a leak.
How do you know it works on unseen data?	Models train on 8 log systems and are tested on 8 others they never saw; templates and names are split so tests use phrasings the models never saw.
What if someone hides the number?	12 disguises are 100% caught (E10); 22 of 26 red-team tricks fully caught (E15).
What if the number has a typo?	Typo repair catches 99.2% of one-typo registered numbers (E12).
Doesn't tokenising break the bank's apps?	No: tokens keep the exact shape, and authorised systems can detokenise, with every call logged.
How is erasure legal if KYC must be kept?	Keys are per purpose. KYC keys are time-locked and destroyed automatically when retention ends; marketing and servicing keys can be destroyed on request. Whether crypto-shredding counts as DPDP erasure still needs a legal opinion; we say so openly.
Is it fast enough?	Not yet for production: 654 lines per second in Python. The port to Rust or vectorised code is on the roadmap with a 10,000 lines per second target.
What are your weaknesses?	Part E3: names at about 77%, two regressions, simulated flow attacks, no real-model gateway test, speed.
Why post-quantum?	Without it, an attacker who recorded a key backup today could un-erase a customer once a quantum computer exists (E17: 100% recovered vs 0% with the hybrid).
Is it a product?	Yes, a backend product for banks: gateway, log cleaner, AI gateway and dashboard, on-premise or managed (G4).

H5. Who explains what

Part	Presenter
Problem, research, theorems (M1, M3, M5), law, PQC	Inbathamizhan S
Normaliser, registries, ledger, immune layer, red team	Aasif Ahmed A
NER, context classifier, calibration	Bharadhwaj M
AI gateway, OCR, speed, results tables	SN Arun Vijay

This split follows the owner pairs; change it as the team prefers.

Appendices

Appendix A. Every document in the repository

Document	What it is
README.md	The front page: problem, core idea, results, live console, run steps, document list
Research document	Problem analysis, baseline, literature survey, gaps, M1 to M5 with theorems, critical review, E1 to E4, sources
AI and immune layers	E5 to E9
Roadmap	Items 1 to 14 with status, owners, effort and learning docs
Normaliser, registries and typo repair	E10 to E12
Ledger, AI gateway and red team	E13 to E16
Post-quantum upgrade	E17 to E19
Website design-system prompt	Brand, colours, type, components and content for a design tool
Frontend guide (folder named after the design tool)	How to design the dashboard, with prompts and the API contract
UI design spec	Every page, component, state, animation and endpoint
Design references	Human-designed sites matched to each page, anti-generic checklist
Standout frontend ideas	The eight demo features, ranked
Console README	Which endpoint feeds each screen
Site research	The five reference websites studied for your design
Artifacts index	Every artifact by thread
Obsidian home, threads index	The vault

Appendix B. The learning docs, and the order to read them

#	Doc	Who	Time
1	Python and Tools	Everyone, first	~6 h
2	Indian Identifiers and Check Digits	Everyone	~5 h
3	Cryptography Basics	Cyber pair (everyone reads section 4)	~7 h
4	Probability for Detection	Everyone	~6 h
5	AI and NLP for Detection	AI/ML pair	~8 h
6	Artificial Immune Systems	Cyber pair	~6 h
7	Data-Protection Law Basics	Everyone	~4 h
8	Web APIs and Proxies	Everyone	~6 h

#	Doc	Who	Time
9	Transformers and Zero-Shot NER	AI/ML pair	~8 h
10	OCR Basics	AI/ML pair	~5 h
11	Frontend Basics	Everyone	~5 h
12	Leakage Ledger, AI Gateway and Red-Teaming	Everyone	~5 h
13	Animation and Visual Explainers	Everyone	~4 h
14	Post-Quantum Cryptography	Cyber pair (everyone reads 1 to 3)	~5 h
15	UI Design Specs and Design Tokens	Everyone	~2 h
16	3D Graphics with three.js	Everyone	~4 h
17	Wiring a Static Design to a Live API	Everyone	~3 h
18	Phones, QR Codes and Speech Input	Everyone	~2 h
19	Translating a Website	Translators especially	~1.5 h
20	Presenter Mode and Offline Replay	Whoever presents	~1 h

Shortest path before a demo: this document, then Learn 2, 7 and 20, then your own pair's docs (cyber: 3 and 6; AI/ML: 5 and 9).

Appendix C. How the work was done

Your decisions that shaped Aran. You set the problem and the bar ("novel, unique, strong, derived from maths or science, real data"); approved the research plan; asked for AI and the biology plan and assigned the pairs; made "learning docs before every plan" and "everything saved, linked and pushed" standing rules; insisted on a real site with no simulated values; pushed for plain-language, visual explanations for non-experts; brought in post-quantum cryptography; designed the final UI yourself; and asked for the stand-out demo features. The research, code, experiments and documents were produced in the project's work threads at your request, each one ending in a commit under your name on `main`.

How the threads ran. Each request became its own thread with a short plan, a status checklist and a final reply with links. Big tasks showed a plan table first and waited for your OK. Work paused four times (the longest for about 11 hours, from 01:34 IST on 17 October) and resumed on its own each time. Work branches were merged into `main` each time; `main` is the only branch that matters.

Where things live.

Place	What is there
GitHub `itzz-inbathamizhans/aran`, branch `main`	Everything: code, docs, artifacts, vault
`D:\Aran` on your laptop	Working copy and Obsidian vault, on `main`

Place	What is there
`D:\Aran-live`, `D:\Aran-demo`	Copies used to run the demo server
`D:\Aran-local-backup-2026-10-17`	24 older local screenshots kept aside on 17 Oct
Project shared folder `artifacts/`	Copies of every artifact, by thread
Notion Second Brain, "Aran" project	Session handoffs

Repository rules. Commits authored as Inbathamizhan S; no tool credits anywhere; every artifact as Markdown and PDF under `artifacts/<topic>/` and linked from the README.

Appendix D. What is real, what is generated, what is simulated

Item	Status
Loghub logs (32,000 lines, 16 systems; BGL labelled set)	**Real**, public, unsanitised production logs
CoNLL-2003, HiNER, Hinglish NER tweets	**Real** public NER corpora
UCI SMS Spam Collection (4,825 "ham" messages)	**Real** public SMS
Aadhaar, card, PAN, mobile, OTP, UPI values	**Generated** with valid checksums; belong to no one
Names and addresses in test lines	**Generated** from random first names, surnames, cities
Bank-style log templates	**Written by the team's threads**; train and test halves share nothing
Bank data-flow telemetry for the immune layer (E8)	**Simulated** (no bank publishes it); the live console replays real Loghub lines as normal traffic
Quantum attack (E17)	**Simulated** by handing the attacker the X25519 private key
LLM provider in E14	A **recorder** that echoes the prompt (the hardest case); real models supported by configuration
Breach figures, regulation figures	**Real**, from the cited sources
Mosca shelf lives and quantum dates	**Assumed** scenarios
Every number on the live console	**Computed live** by the server; tour figures are read from the result files

Appendix E. Key numbers to remember

Number	Meaning
67,481 → 6	OTP false alarms on 32,000 real lines: plain pattern vs Aran
0 of 20,000	Tokens wrongly flagged as real
100%	Leaked dumps traced to the right sink among 40
1,000	Candidates left by "first 8 + last 4" card masks (10 bits)
46.1% → 91.0%	Sensitive values removed: regex vs Aran (no registries)
2,506 → 60	False alarms on 16,000 real lines: regex vs calibrated detector
100% at 0.6%	Immune layer: simulated attacks caught, false-alarm rate

Number	Meaning
8.3% → 100%	Identifiers hidden 12 ways, caught
99.2%	One-typo identifiers caught
14,175 → 0	Customers a vendor could narrow below 20 bits by joining systems
100% → 7.8%	Raw values reaching an LLM provider (every number type 0%)
22 of 26	Red-team hiding tricks fully caught
654	Lines per second today
100% → 0%	Erased values recovered after a simulated quantum break, classical vs hybrid
₹250 crore	Maximum DPDP penalty for failed safeguards
13 May 2027	DPDP Rules duties begin